

TinyML and Audio – Lab #1

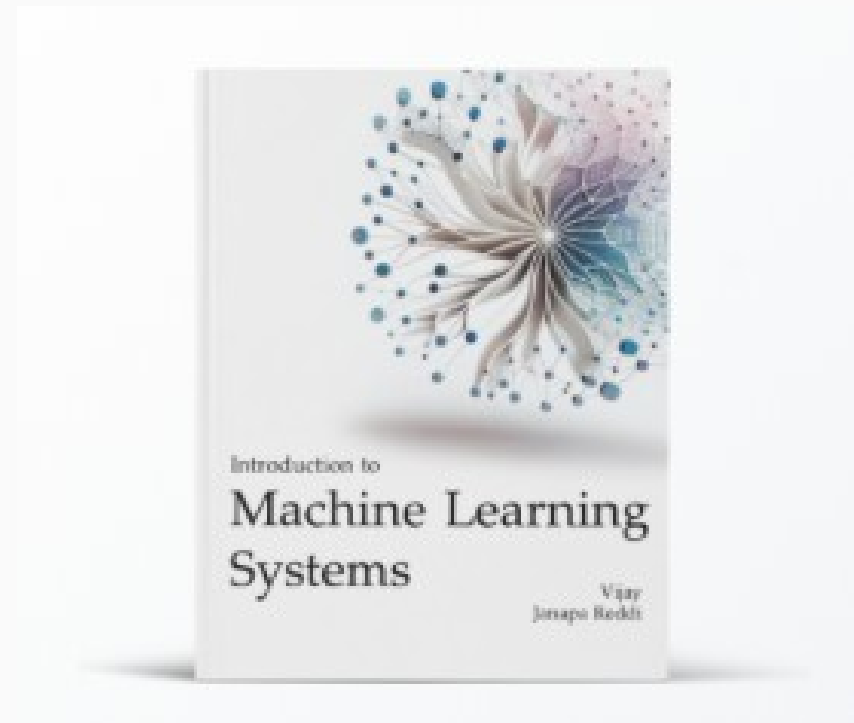
Marco Zennaro, PhD

- The Abdus Salam International Centre for Theoretical Physics – Trieste - Italy
- KIC – Kobe - Japan
- mzennaro@ictp.it, zennaro@kic.ac.jp
- [@marcozennaro](https://www.youtube.com/channel/UCs08333333333333333333)

Thanks

The slides are based on
Prof. Marcelo Rovai chapters
in the book "Machine
Learning Systems"

<https://www.mlsysbook.ai>



**"What we have to learn to do,
we learn by doing."**

Aristotle

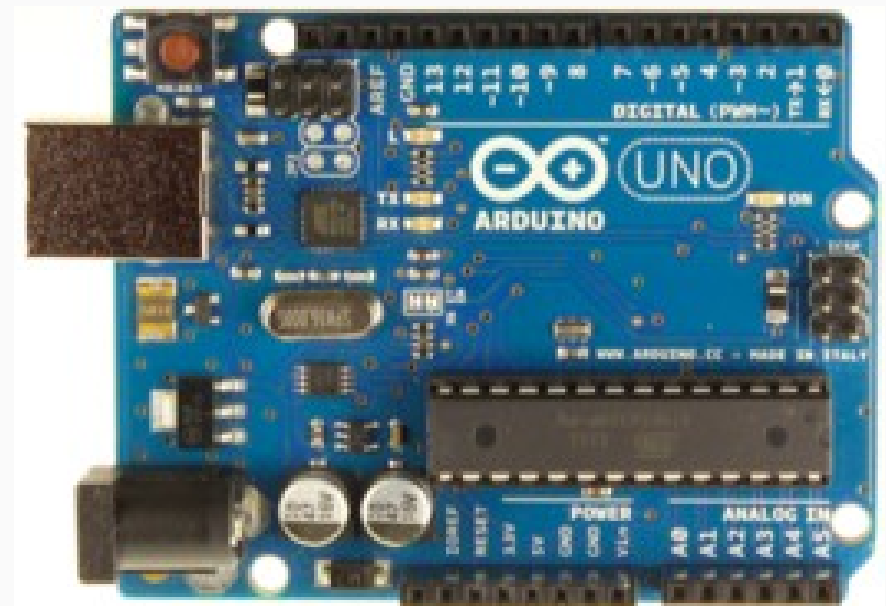


Step Zero

Clean up your desks! :)

Arduino

- Arduino is an open-source electronics prototyping platform based on flexible, easy-to-use hardware and software.
- <http://www.arduino.cc>
- Arduino is:
 - Inexpensive
 - Quite easy to learn
 - Flexible
 - Good for sensing and controlling
 - Great for use in education



Arduino software

- The programming language is based on Wiring and in terms of syntax (almost) identical to C++.
- This is the template of a basic Arduino program:

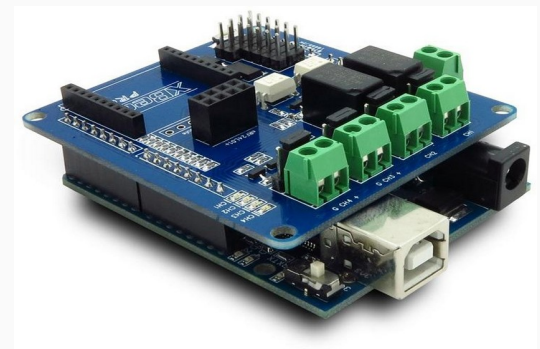
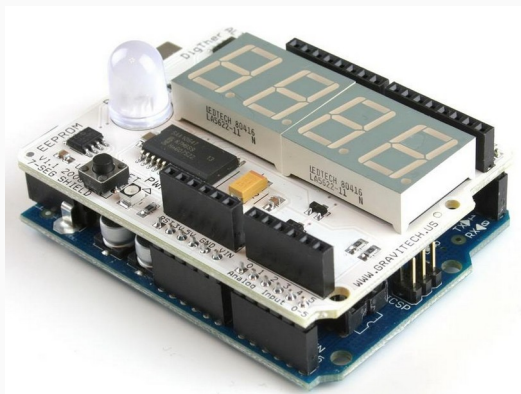
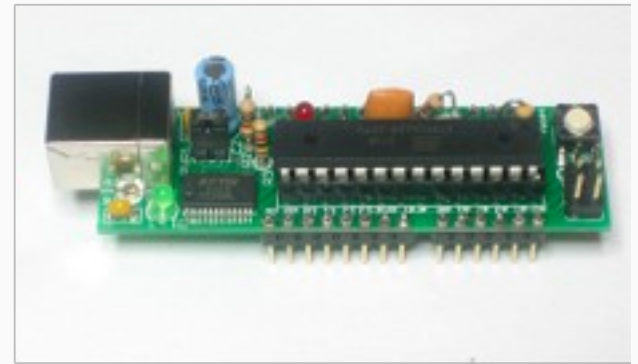
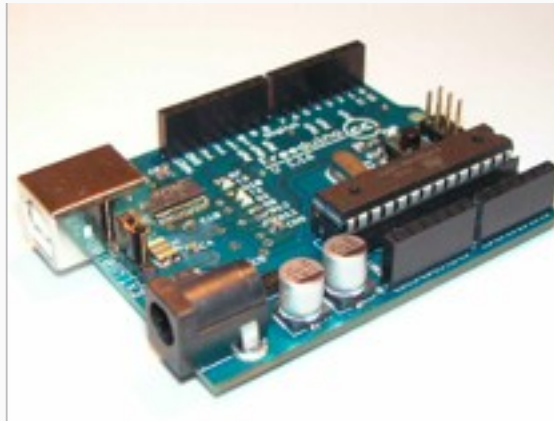
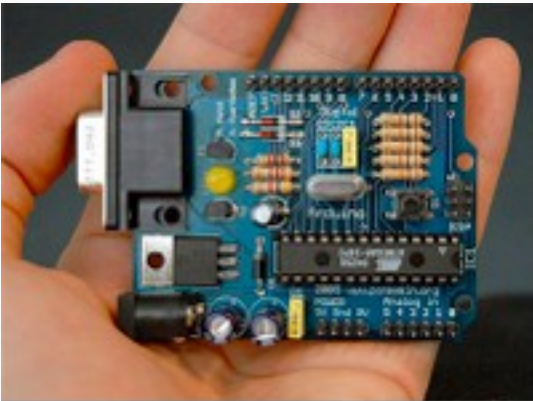
```
void setup()  
{  
  Initialize variables, open USB, open WiFi, etc  
}
```

**SETUP
(once)**

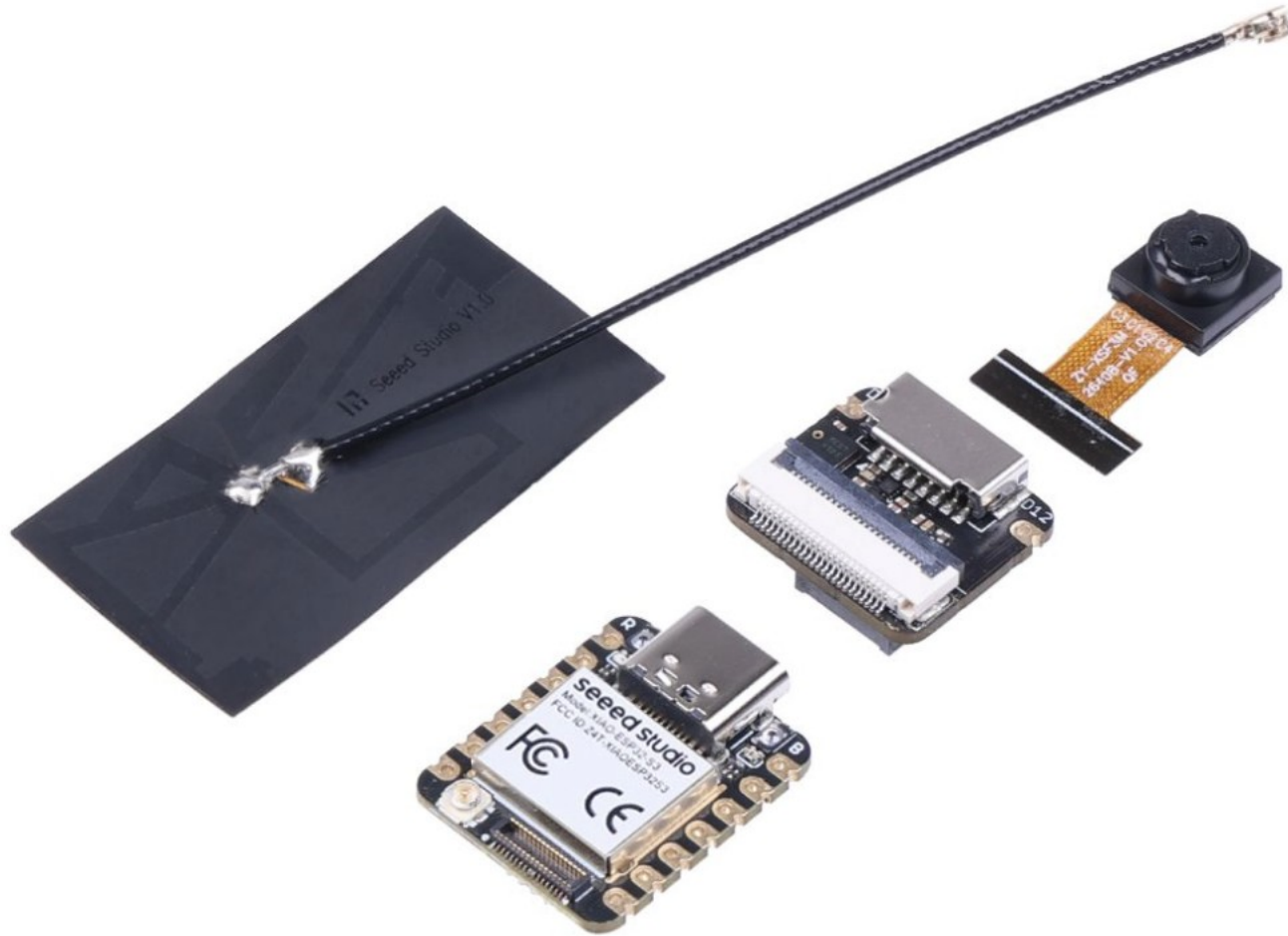
```
void loop()  
{  
  Perform some action  
  
  Wait for a certain number of msecs or wait for  
  an alarm  
}
```

**LOOP
(forever)**

Arduino compatible boards



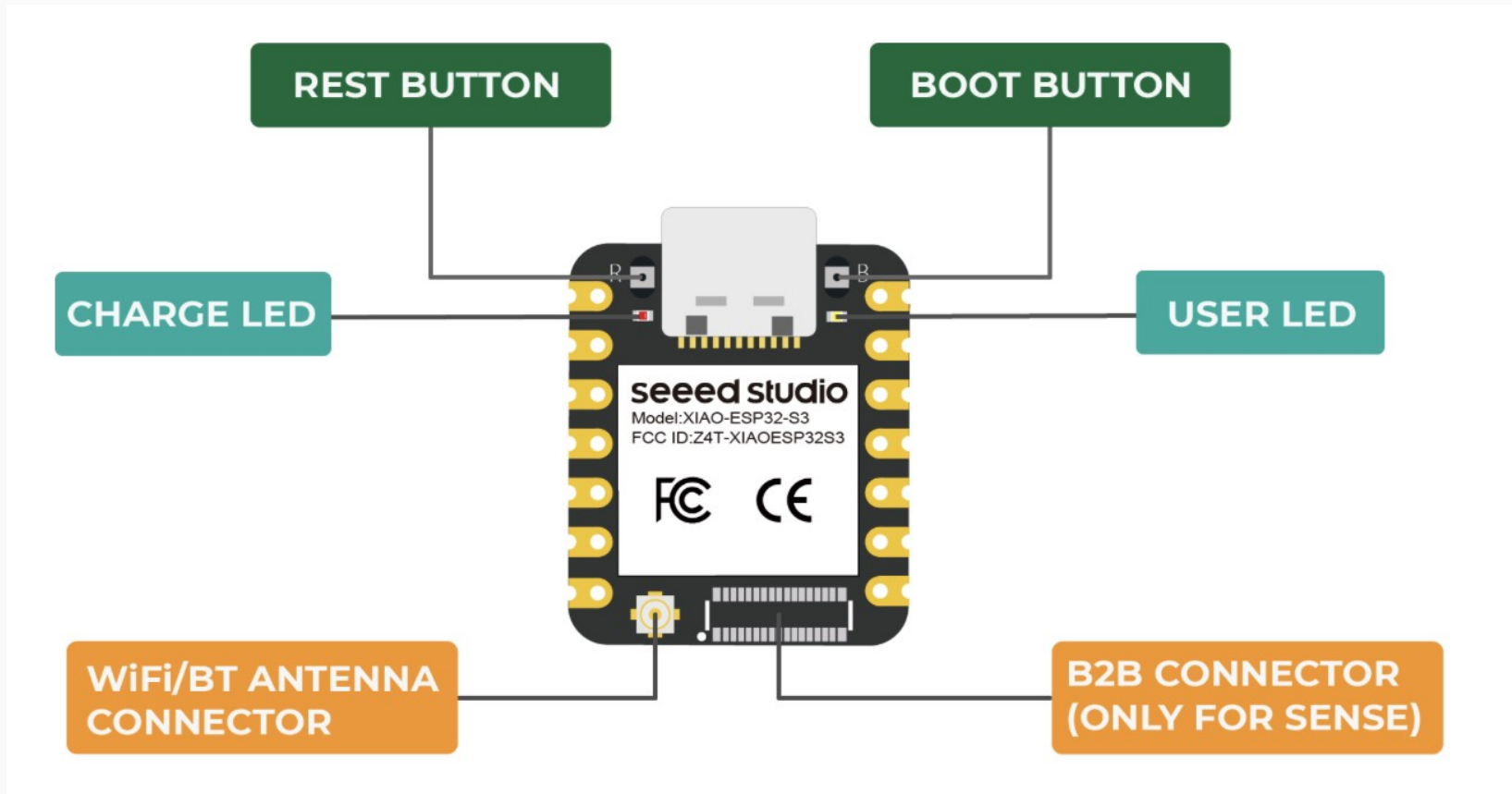
Xiao board



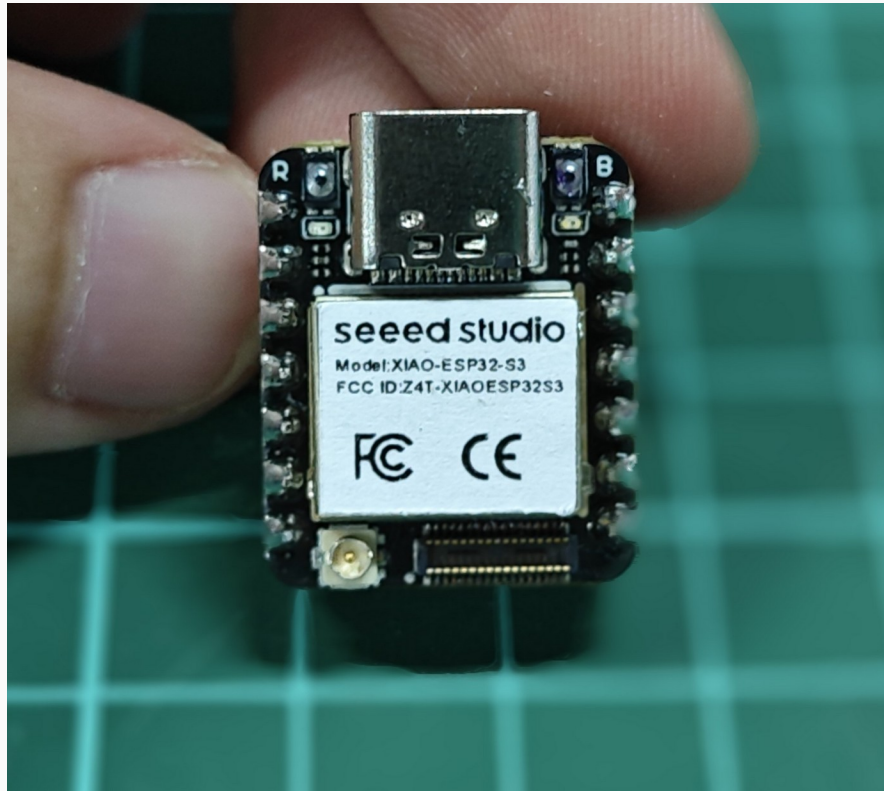
Xiao board

- **Powerful MCU Board:** Incorporate the ESP32S3 32-bit, dual-core, Xtensa processor chip operating up to 240 MHz, mounted multiple development ports, Arduino / MicroPython supported
- **Advanced Functionality:** Detachable OV2640 camera sensor for 1600*1200 resolution, compatible with OV5640 camera sensor, integrating additional digital microphone
- **Great Memory for more Possibilities:** Offer 8MB PSRAM and 8MB FLASH, supporting SD card slot for external 32GB FAT memory
- **Outstanding RF performance:** Support 2.4GHz Wi-Fi and BLE dual wireless communication, support 100m+ remote communication when connected with U.FL antenna
- **Thumb-sized Compact Design:** 21 x 17.5mm, adopting the classic form factor of XIAO, suitable for space-limited projects like wearable devices

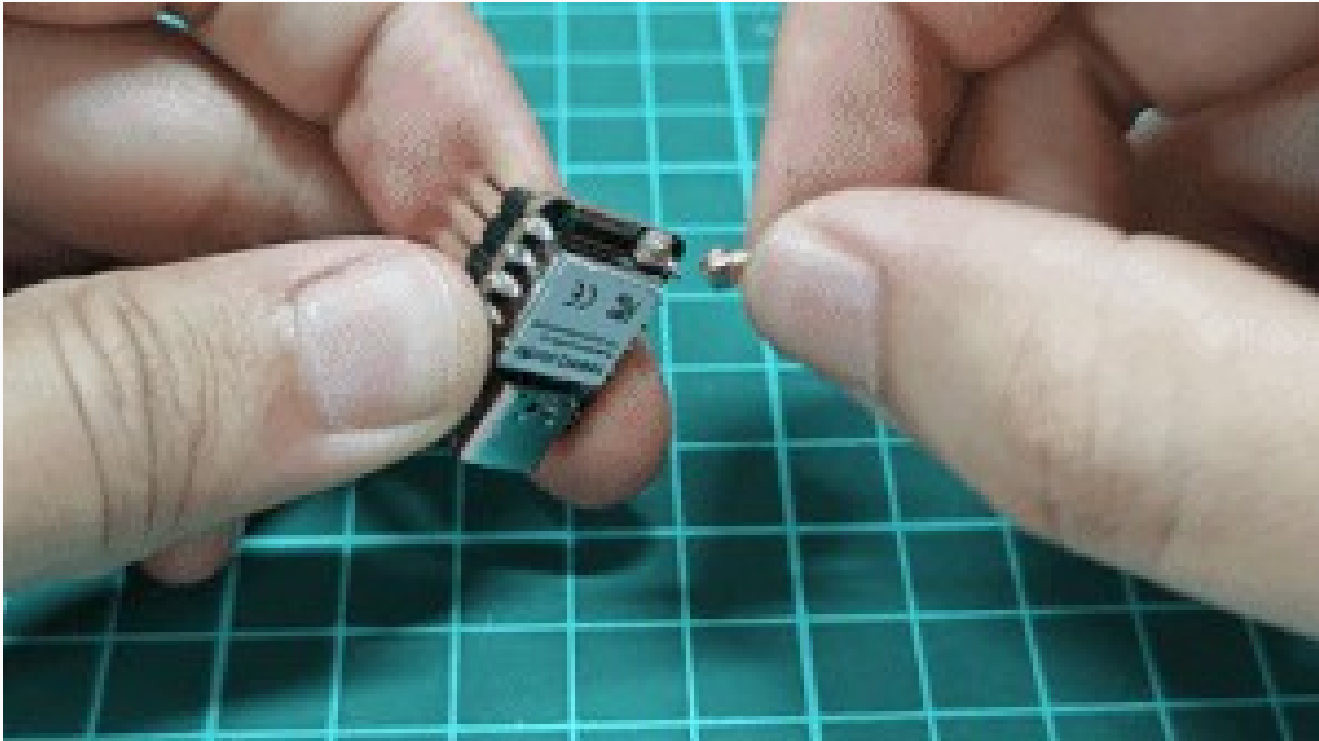
Xiao board



Xiao board



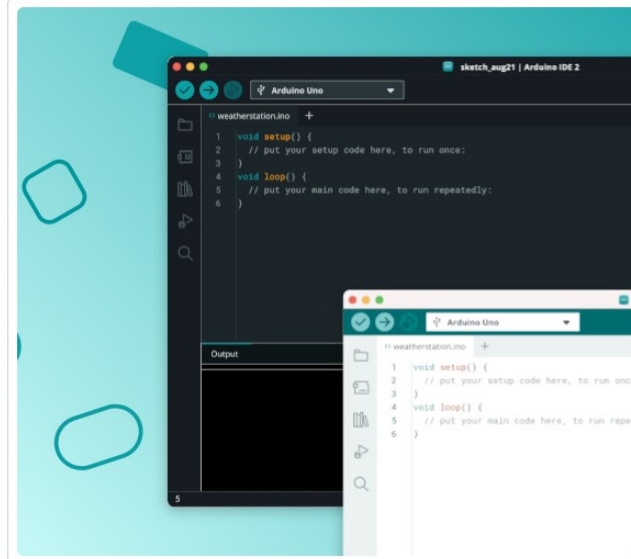
Xiao board



The correct way to install the antenna is to put one side of the antenna connector into the connector block first, then press down a little on the other side, and the antenna will be installed.

Arduino IDE

- IDE= Integrated development environment
- Arduino IDE is Open Source
- Go to: <https://www.arduino.cc/en/software>



Arduino IDE 2.3.6

[Release notes](#)

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

macOS Intel 10.15 Catalina or newer (64-bit)

[DOWNLOAD](#)



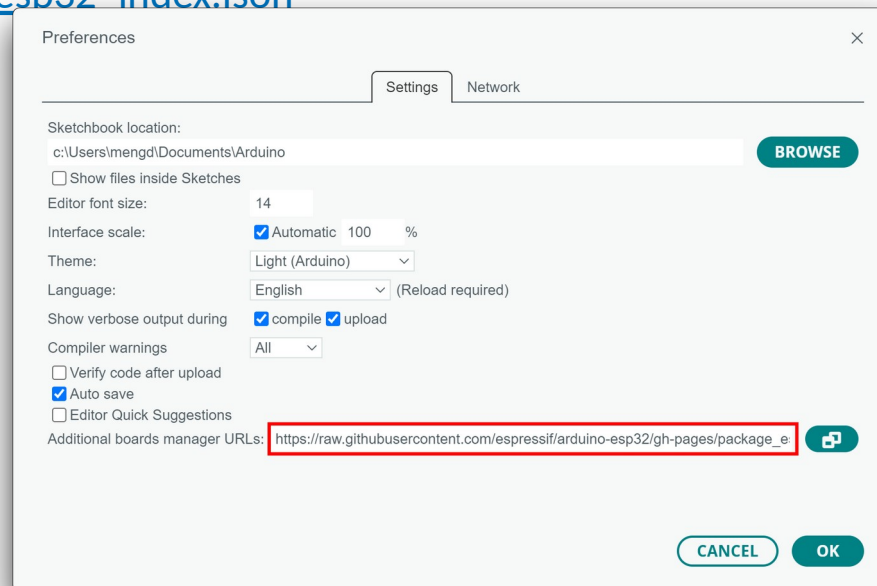
Nightly Builds

Download a preview of the incoming release with the most updated features and bugfixes.

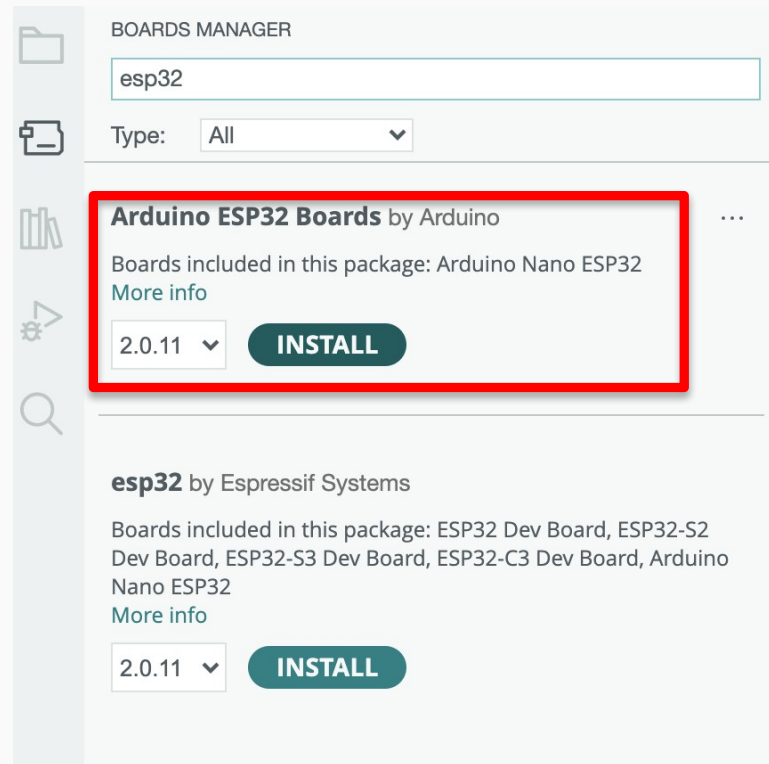
The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Arduino IDE

- Launch the Arduino application. Add ESP32 board package to your Arduino IDE.
- Navigate to Settings, and fill "Additional Boards Manager URLs" with the url below:
 - 0 https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

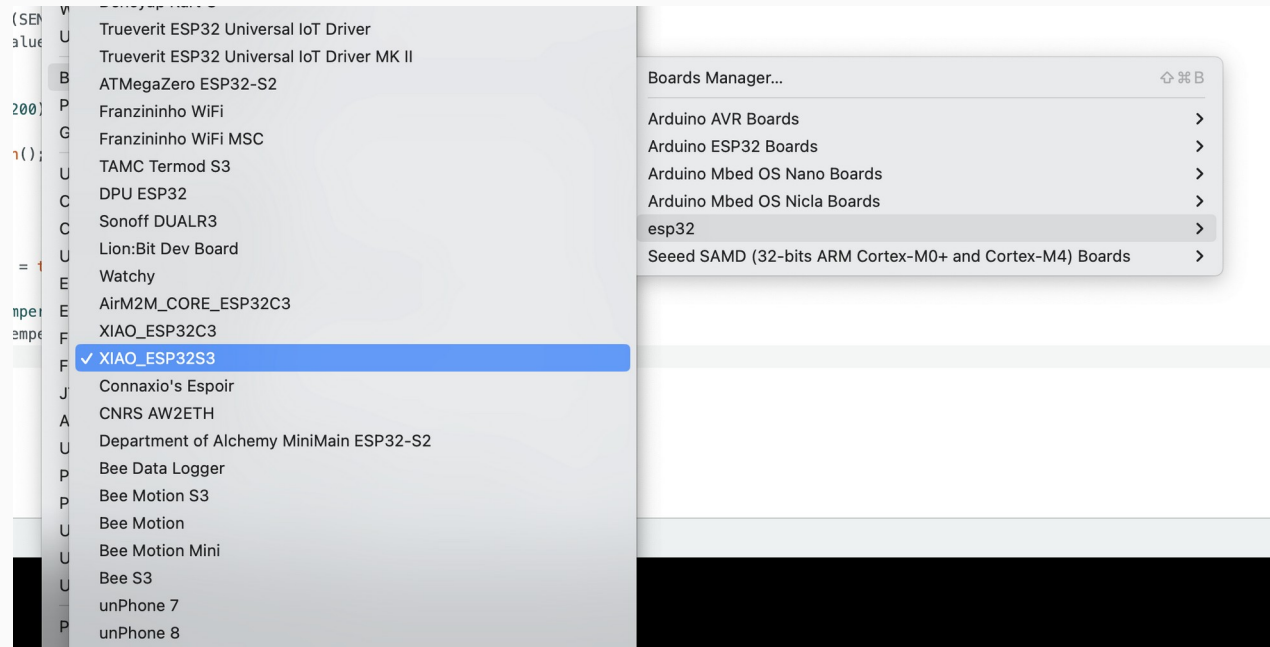


- Navigate to Tools > Board > Boards Manager..., type the keyword esp32 in the search box, select the latest version of esp32, and install it.



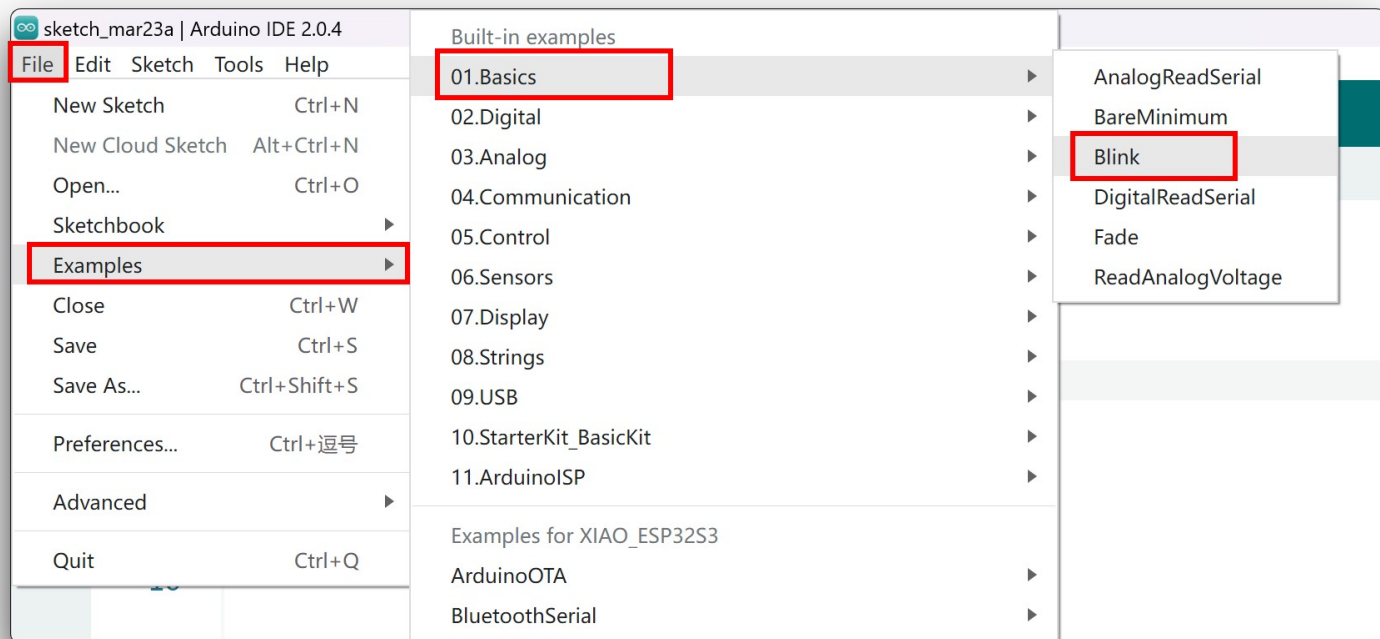
On top of the Arduino IDE, you can select the port directly. In Windows, this is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports).

- Also, search for **esp32** in the development board on the right. Select **XIAO_ESP32S3**.



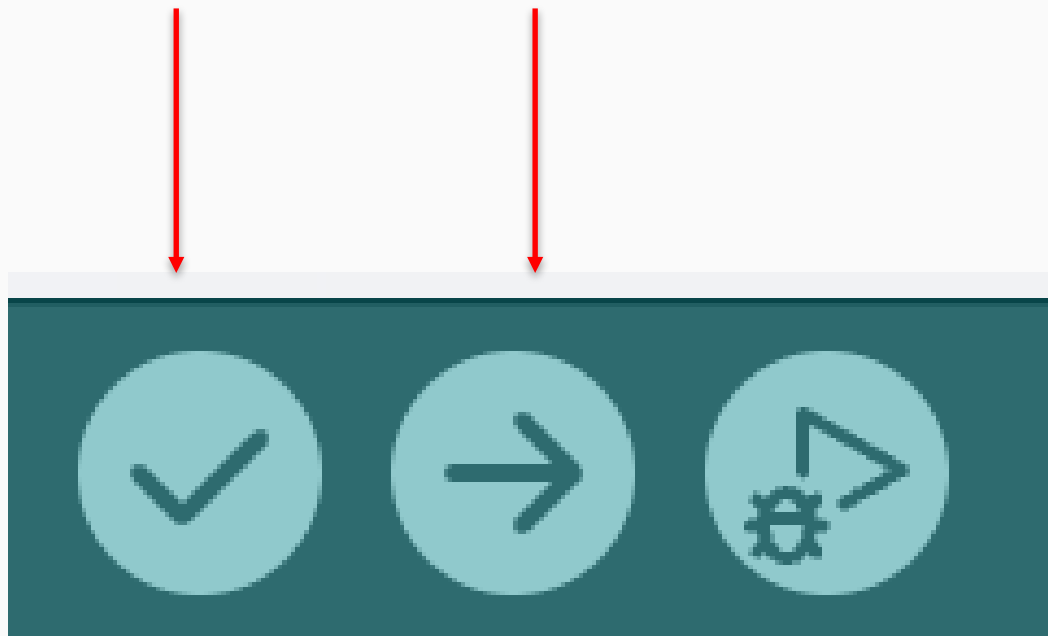
Blink

- Next, let's take the simplest **Blink** program as an example and perform the first blink for your XIAO ESP32S3!
- Launch the Arduino application.
- Navigate to File > Examples > 01.Basics > Blink, open the program.

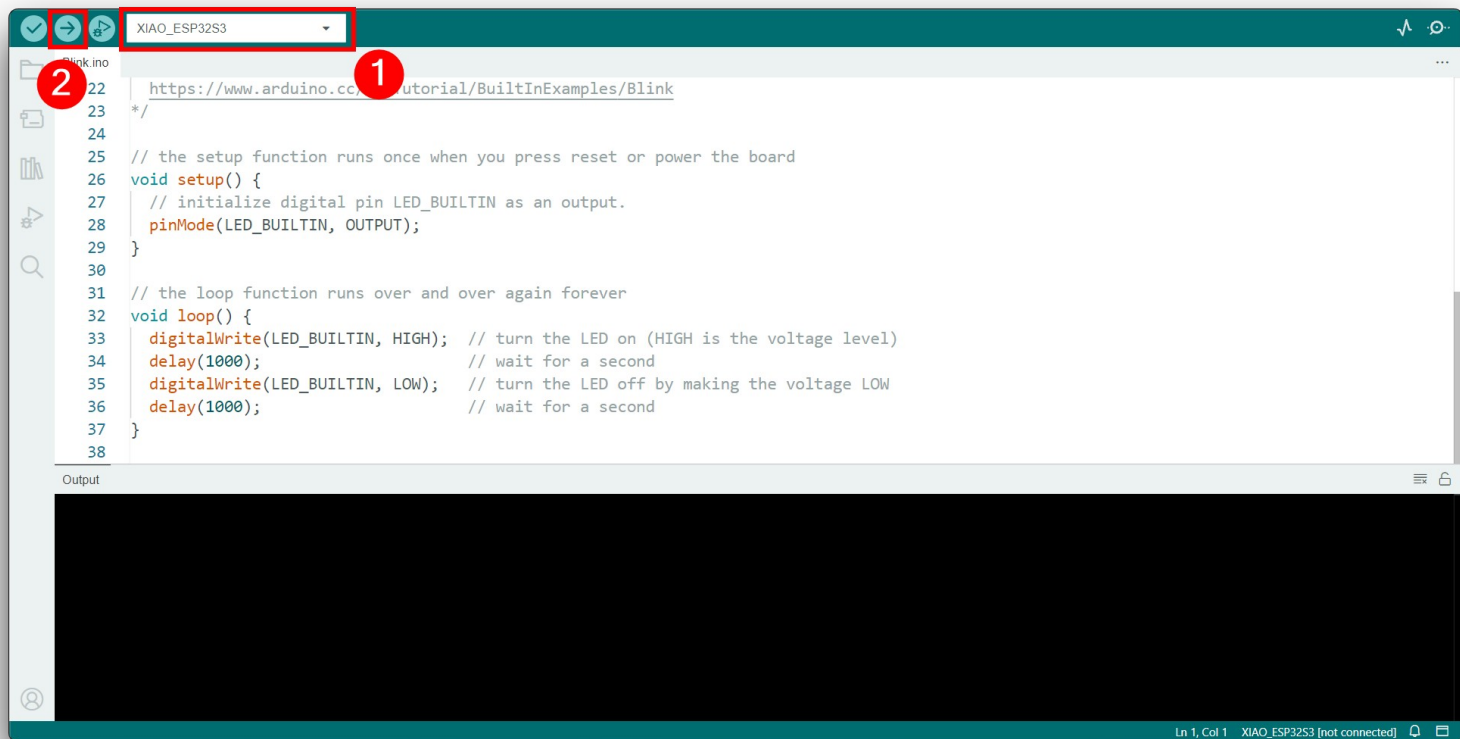


Compile

Upload

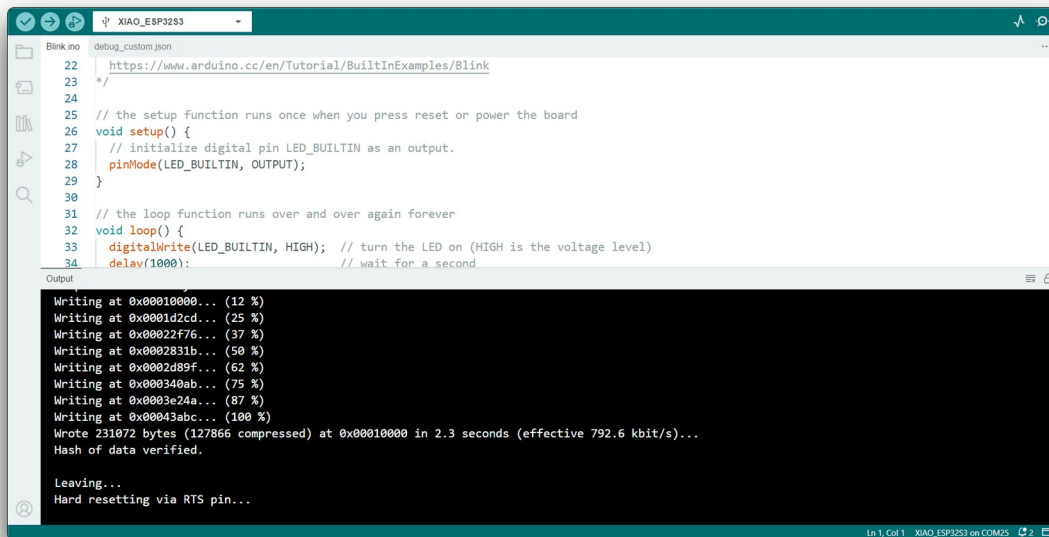


- Select the board model to XIAO ESP32S3, and select the correct port number to upload the program.



Arduino IDE

- Once the program is successfully uploaded, you will see the following output message and you can observe that the orange LED on the right side of the XIAO ESP32S3 is blinking.



The screenshot shows the Arduino IDE interface. The top toolbar includes icons for file operations, compilation, and uploading. The main editor displays the 'Blink.ino' file with the following code:

```
22  https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
23  */
24
25  // the setup function runs once when you press reset or power the board
26  void setup() {
27    // initialize digital pin LED_BUILTIN as an output.
28    pinMode(LED_BUILTIN, OUTPUT);
29  }
30
31  // the loop function runs over and over again forever
32  void loop() {
33    digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
34    delay(1000); // wait for a second
```

The output window at the bottom shows the upload progress and completion message:

```
Writing at 0x0010000... (12 %)
Writing at 0x001d2cd... (25 %)
Writing at 0x0022f76... (37 %)
Writing at 0x002831b... (50 %)
Writing at 0x002d89f... (62 %)
Writing at 0x00340ab... (75 %)
Writing at 0x003e24a... (87 %)
Writing at 0x0043abc... (100 %)
Wrote 231072 bytes (127866 compressed) at 0x0010000 in 2.3 seconds (effective 792.6 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

The status bar at the bottom indicates 'Ln 1, Col 1 XIAO_ESP32S3 on COM25'.



- NOTE: If you check the code, you will see that the LED will only turn OFF when the user LED pin on the XIAO ESP32S3 is set to a high level, and it will only turn ON when the pin is set to a low level.
- Exercise:
 - Get acquainted with the IDE 
 - Make the LED blink for a different amount of time Make the LED blink as fast as possible! 

KWS (KeyWord Spotting)

Introduction

Personal Assistant

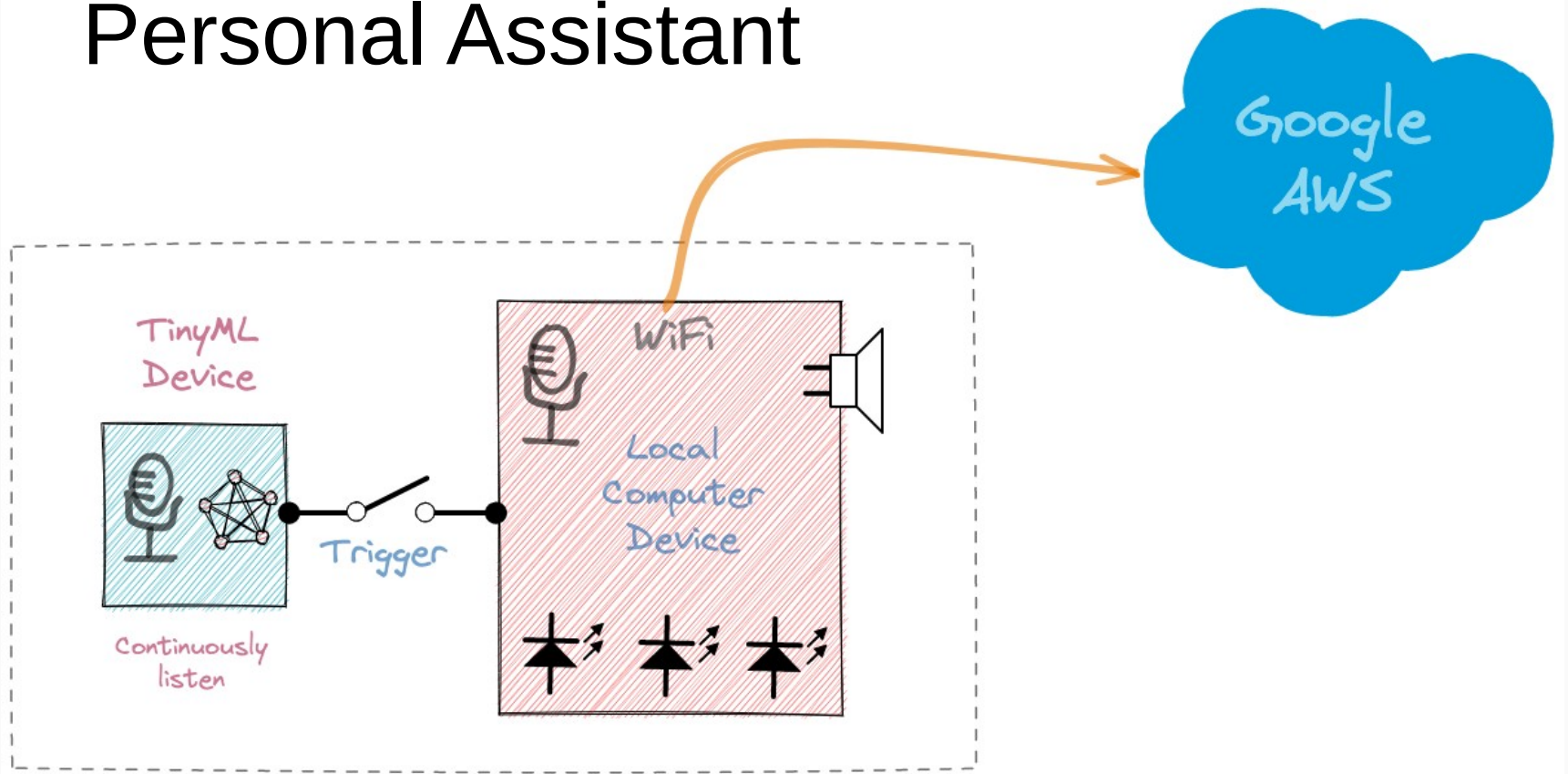


There are an estimated **8.4 billion devices with AI personal assistants** (like Alexa, Siri, Google Assistant, etc.) in use globally as of the end of 2024!

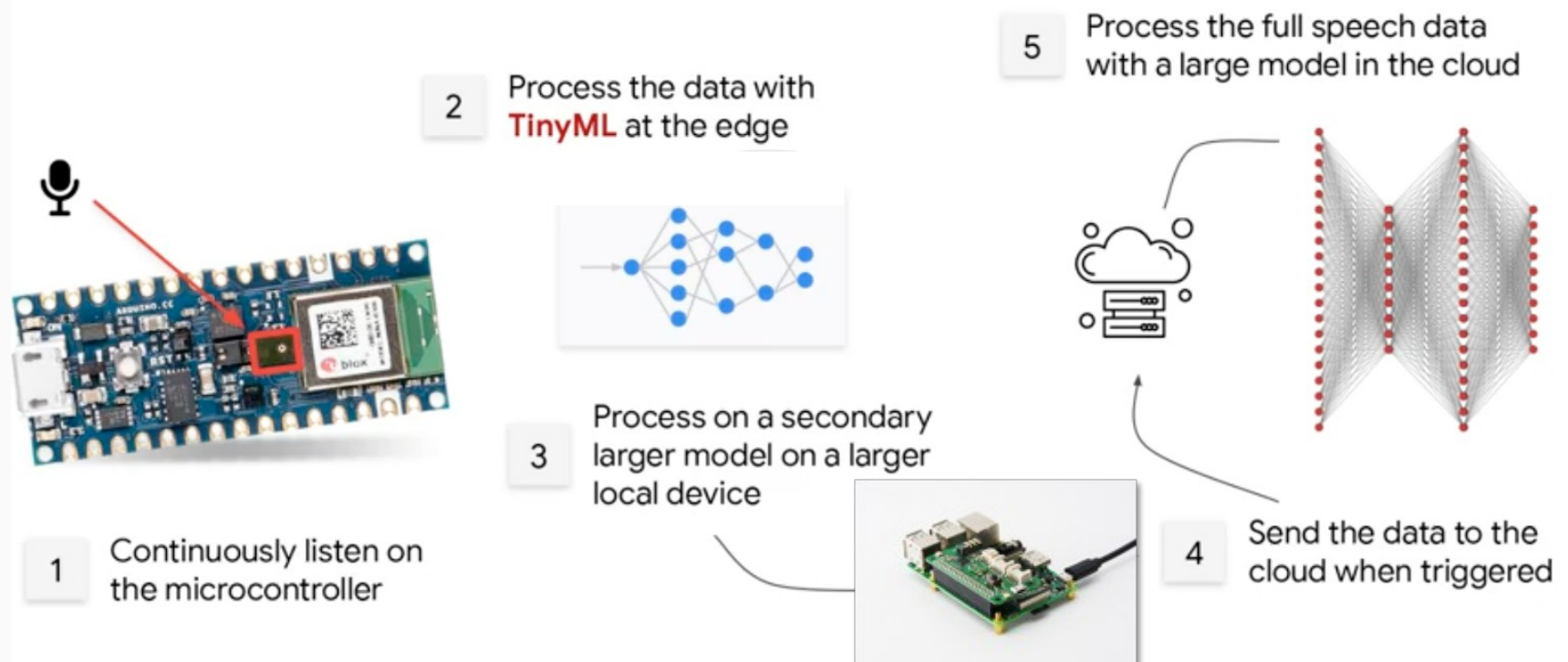
Keyword Spotting v. General Speech Recognition

- **Keyword spotting** is one of the most successful examples of **TinyML**
 - Low-power, continuous, on-device
 - Common Voice SWTS^{*} expands keyword spotting to more languages
 - * Single Word Target Segment
- **General ASR**^{*} still requires **larger, power-hungry models**
 - But it can run on mobile devices (offline dictation on smartphones)
 - * Automatic Speech Recognition

Personal Assistant



“Cascade” Detection: multi-stage model

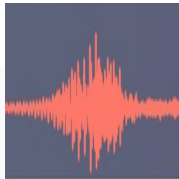


KeyWord Spotting (KWS) - Inference

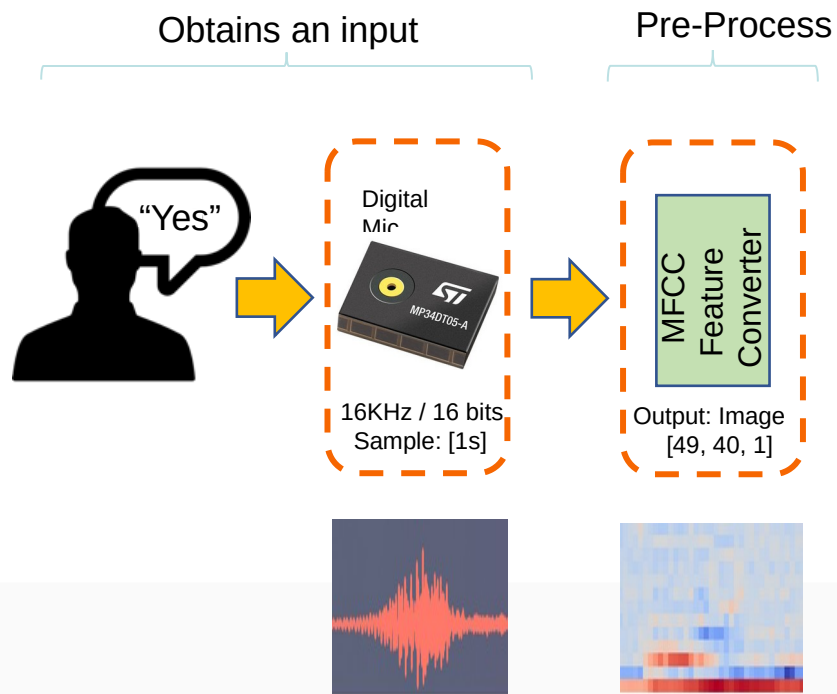
Obtains an input



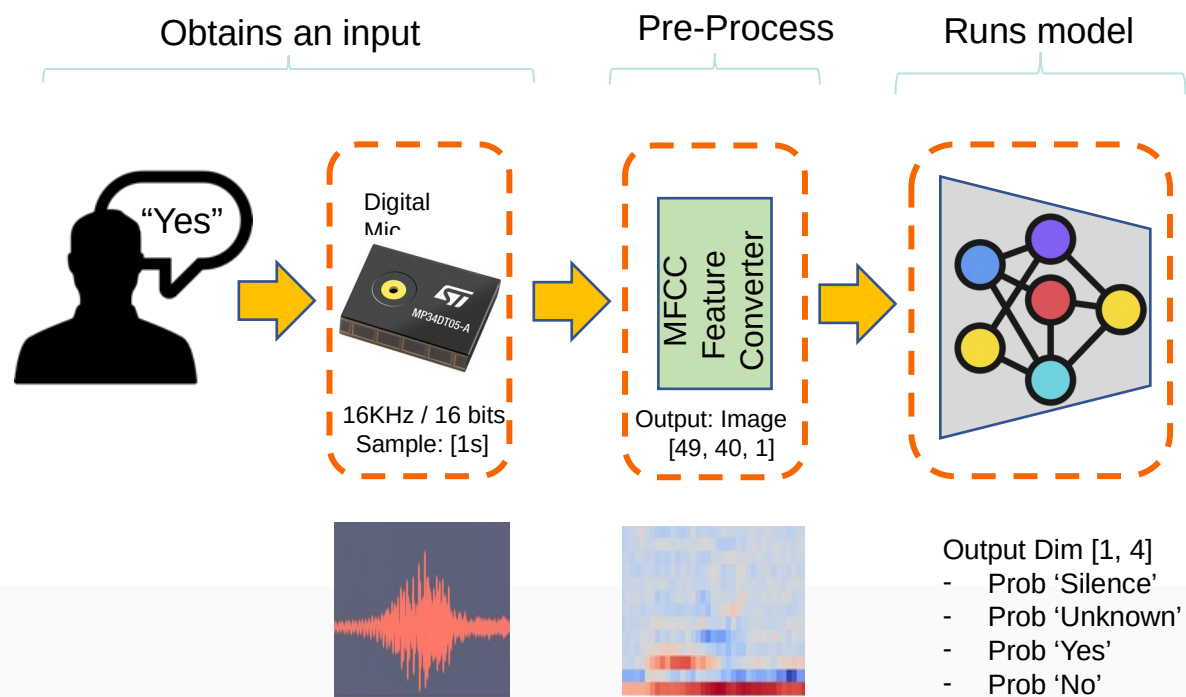
16KHz / 16 bits
Sample: [1s]



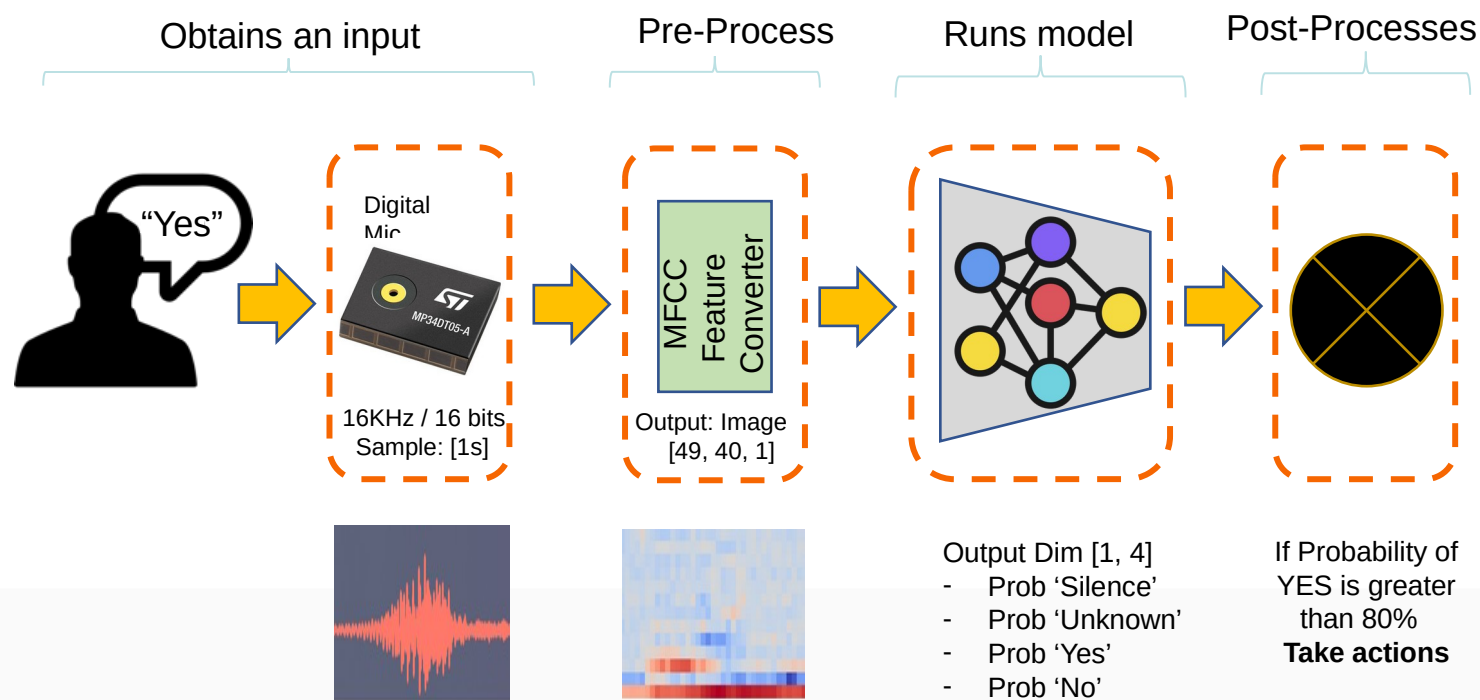
KeyWord Spotting (KWS) - Inference



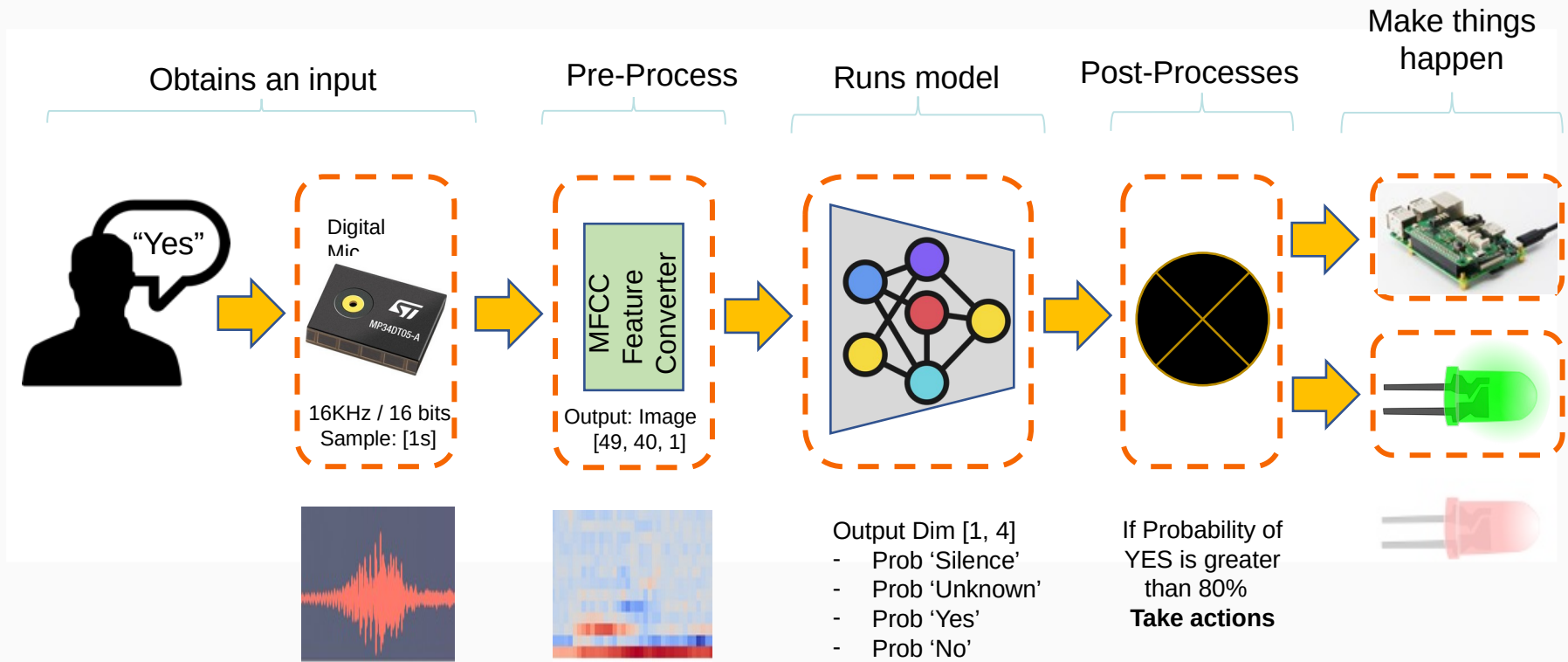
KeyWord Spotting (KWS) - Inference



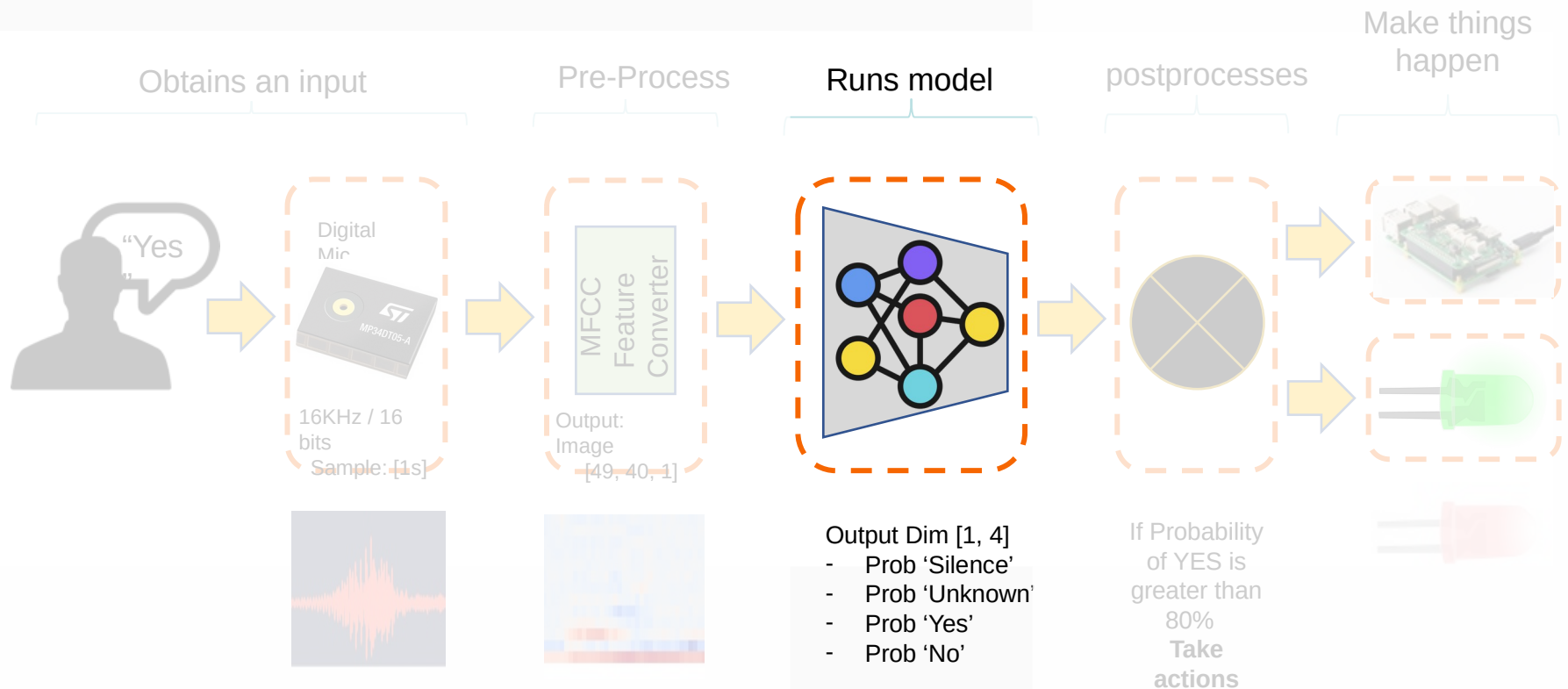
KeyWord Spotting (KWS) - Inference



KeyWord Spotting (KWS) - Inference



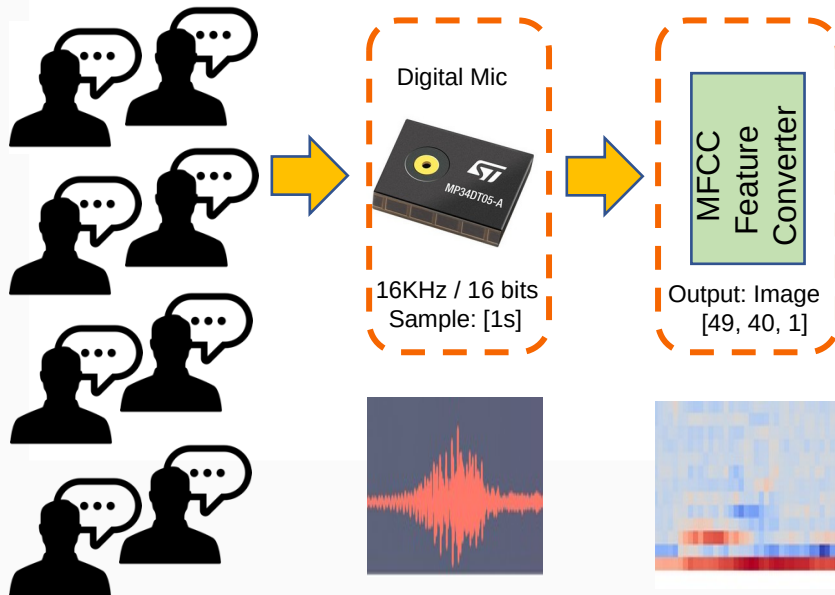
KeyWord Spotting (KWS) - **Model**



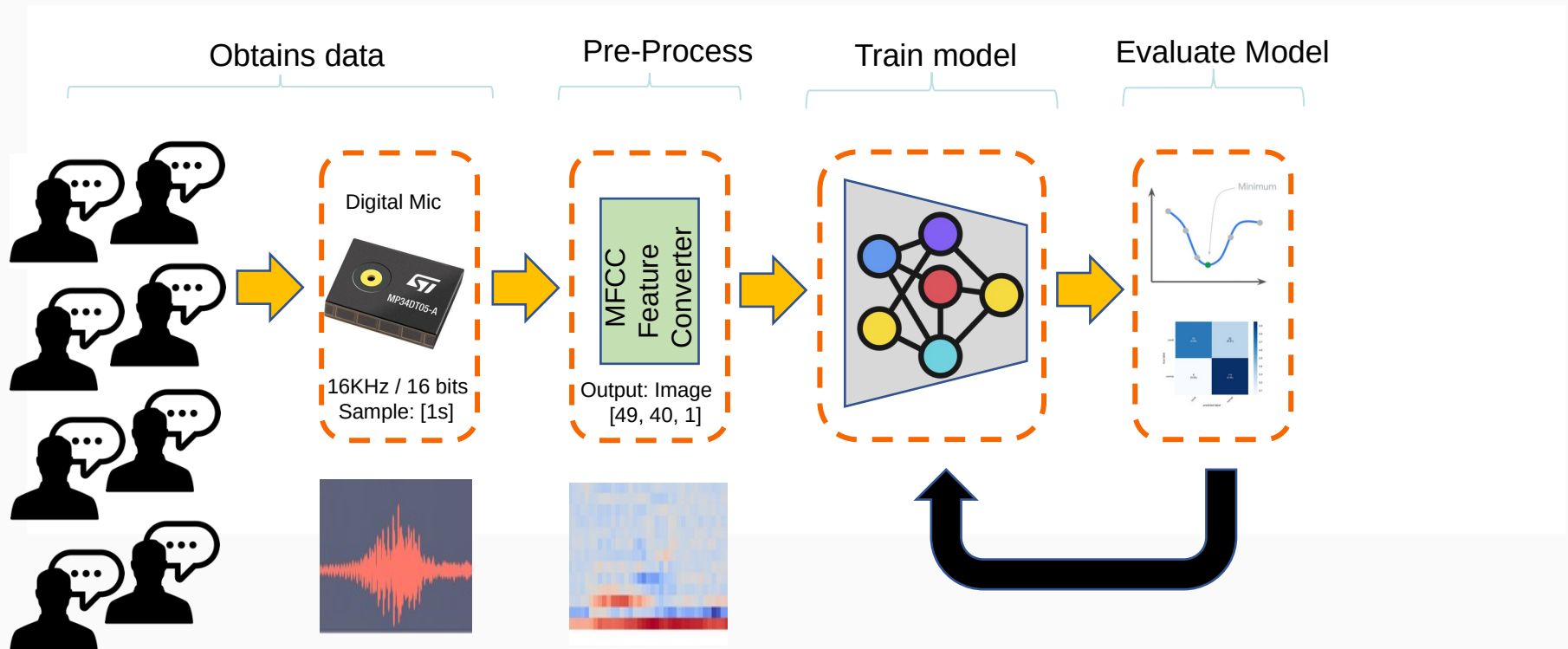
KeyWord Spotting (KWS) – Create Model (Training)

Obtains data

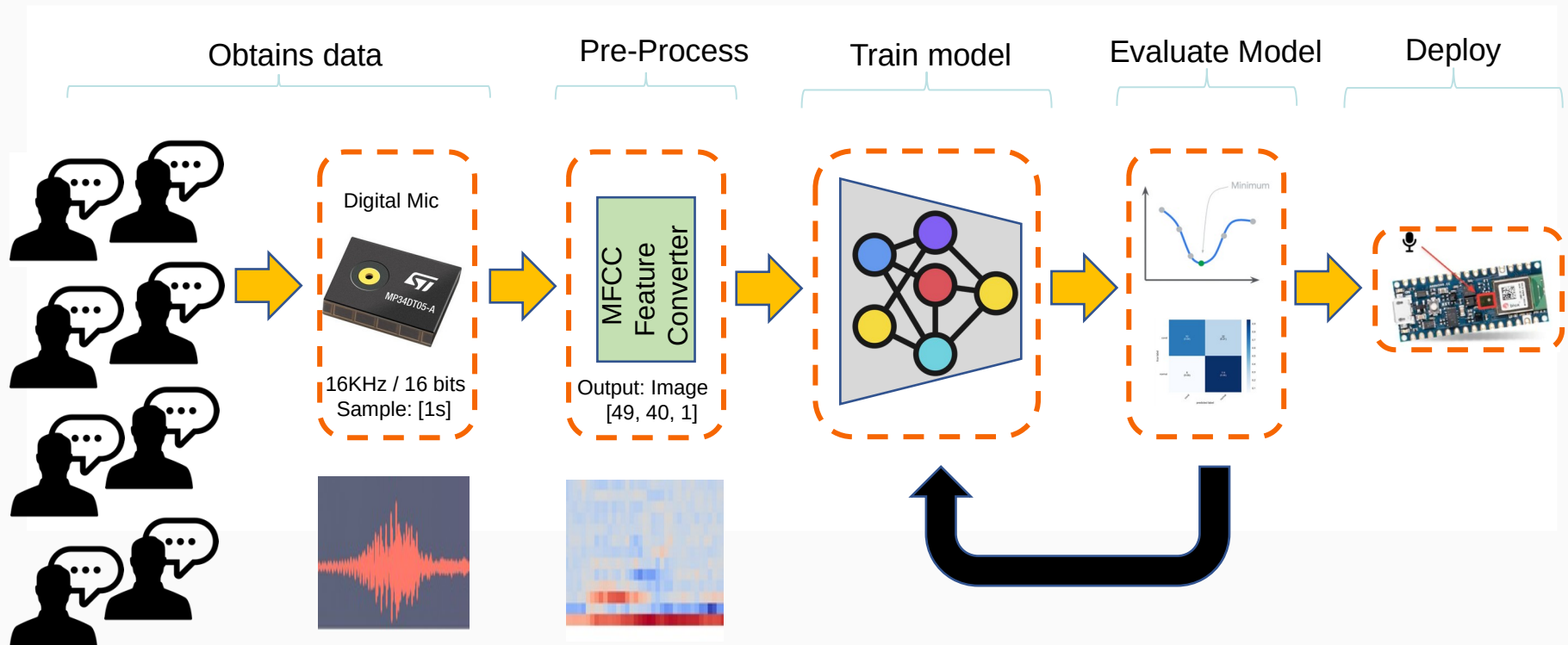
Pre-Process



KeyWord Spotting (KWS) – Create Model (Training)

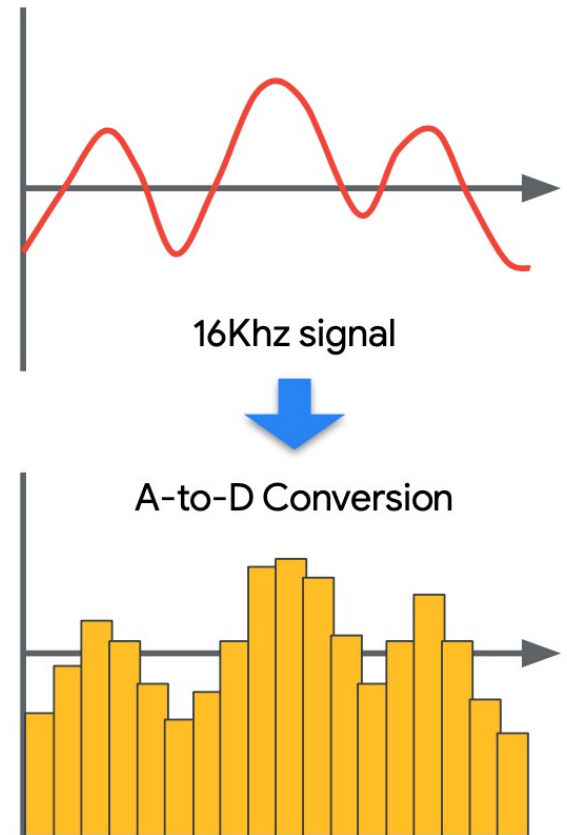
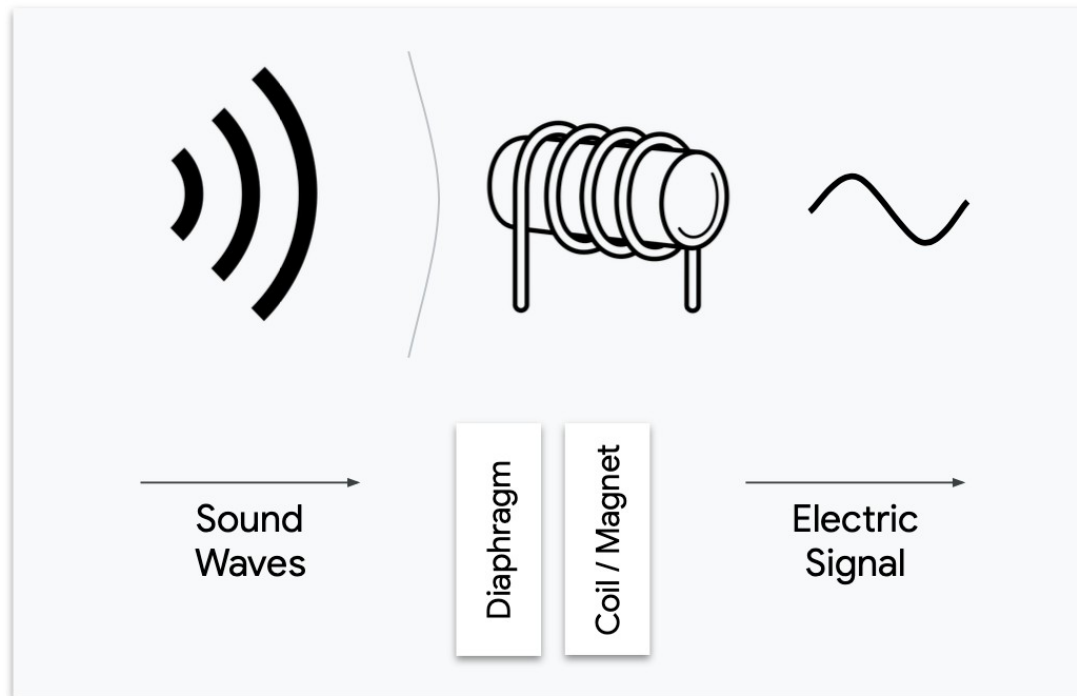


KeyWord Spotting (KWS) – Create Model (Training)



KWS Data Collection & Pre-Processing

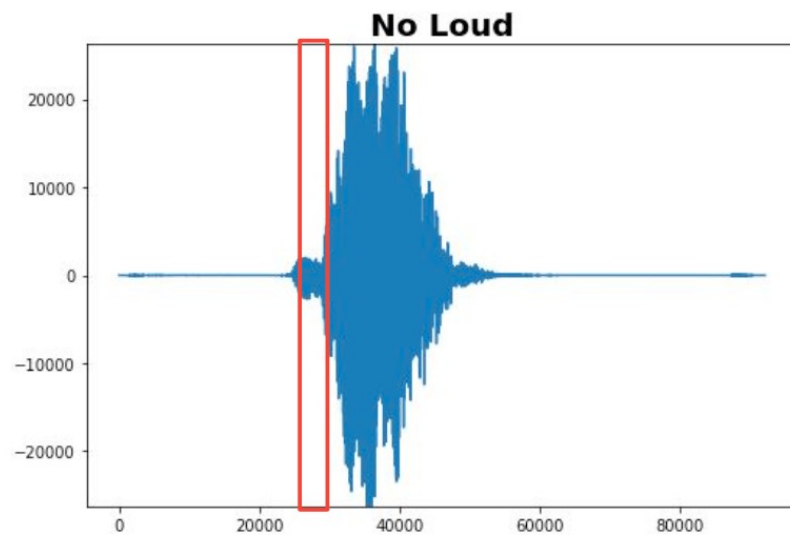
Sensor Data



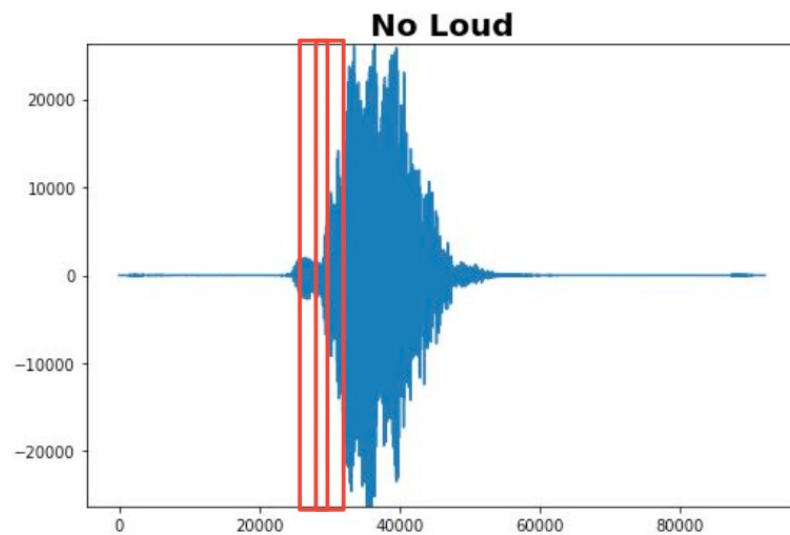
Signal Components?



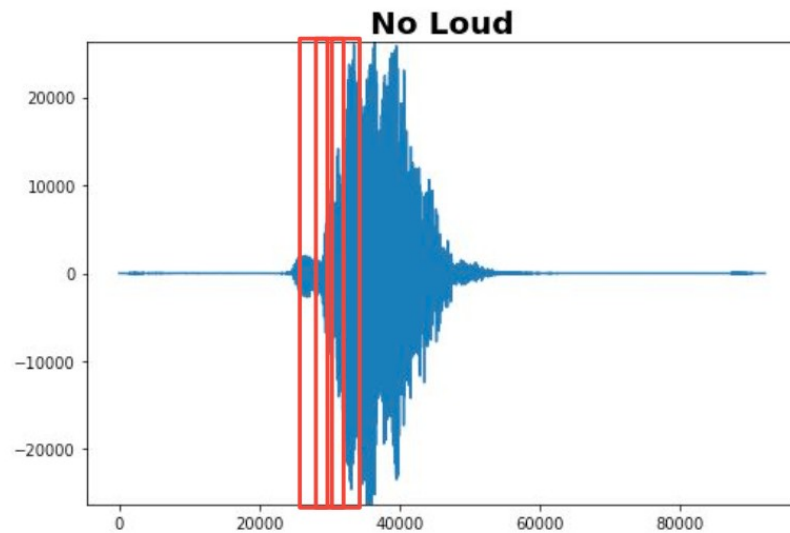
Data Preprocessing



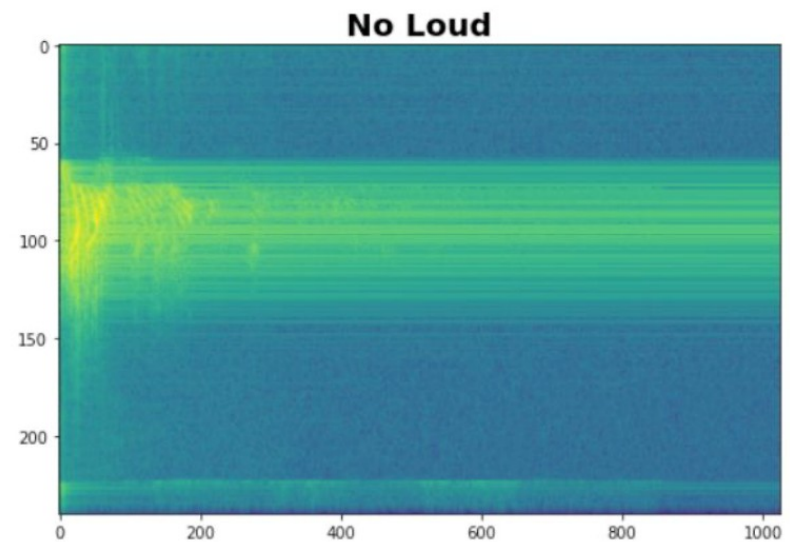
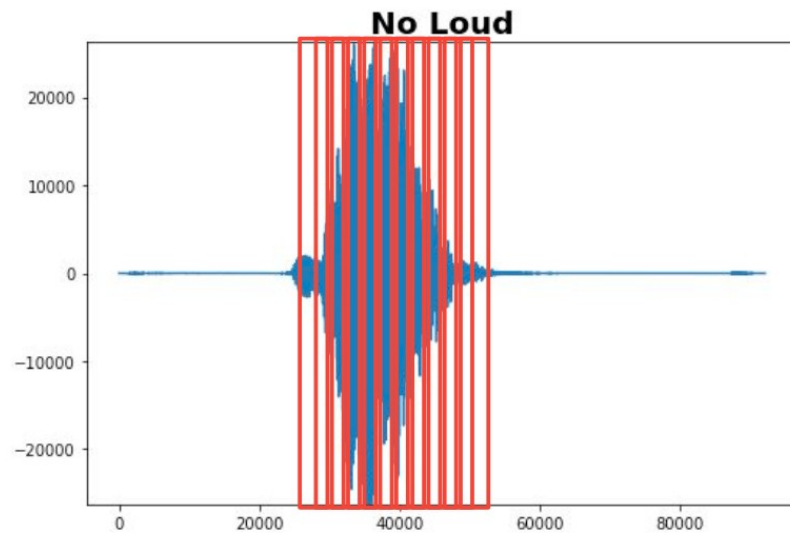
Data Preprocessing



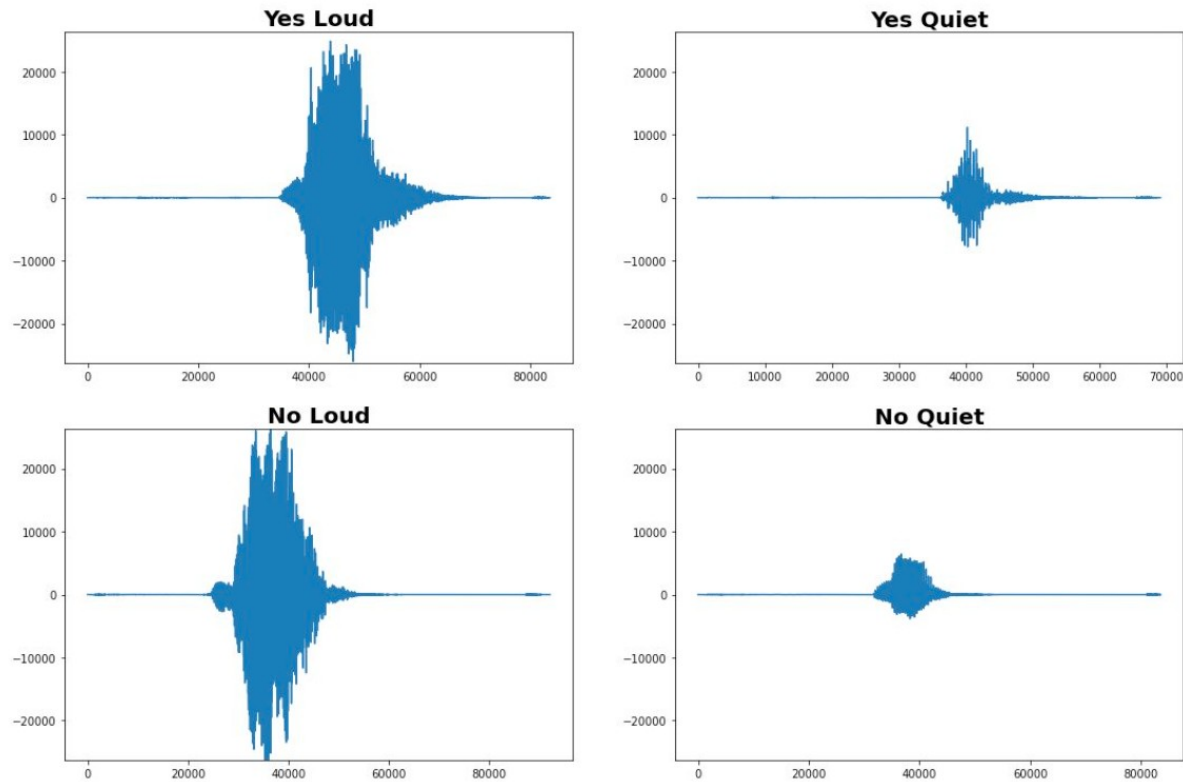
Data Preprocessing



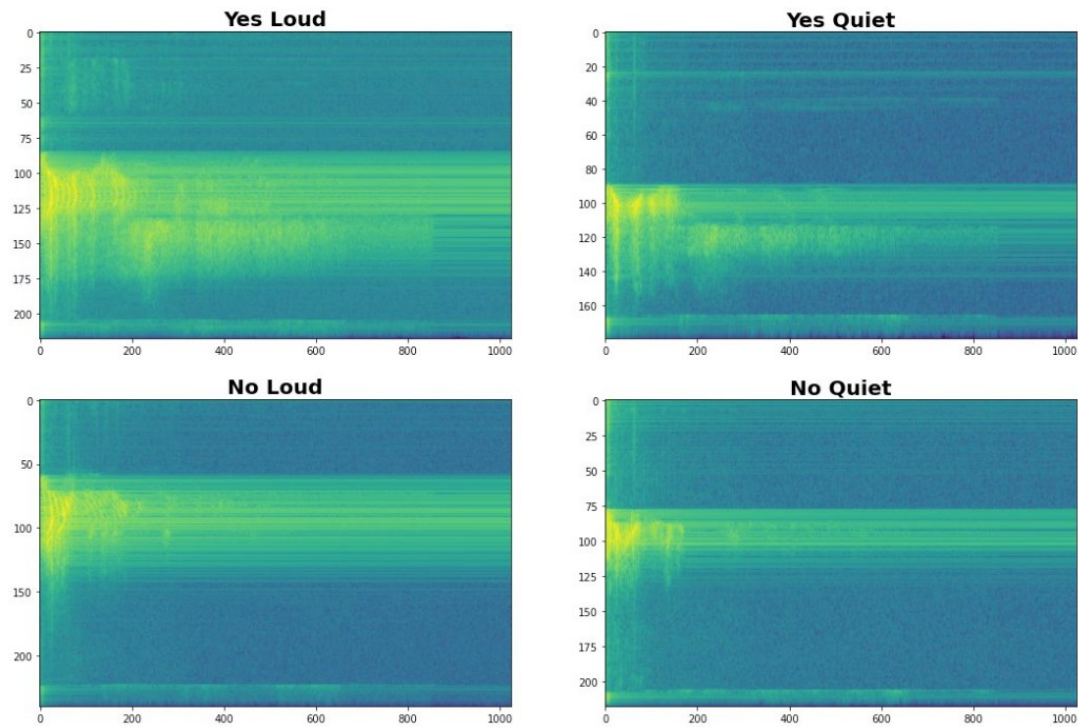
Data Preprocessing: Spectrograms



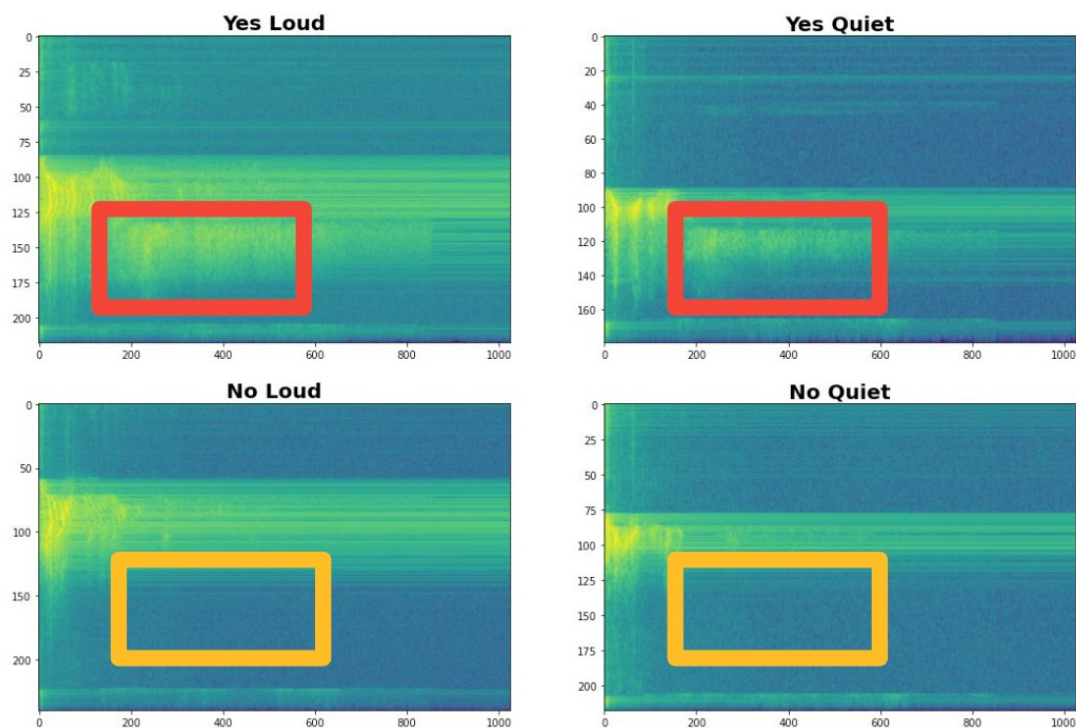
Data Preprocessing: Spectrograms

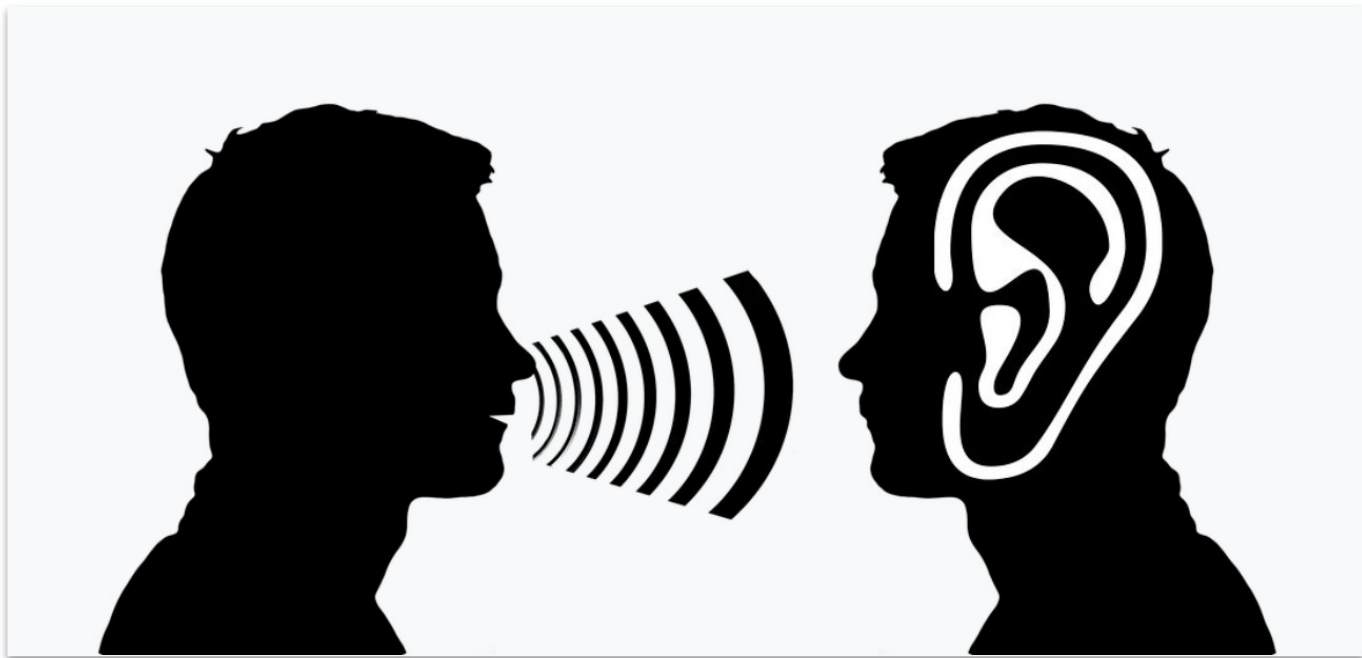


Data Preprocessing: Spectrograms



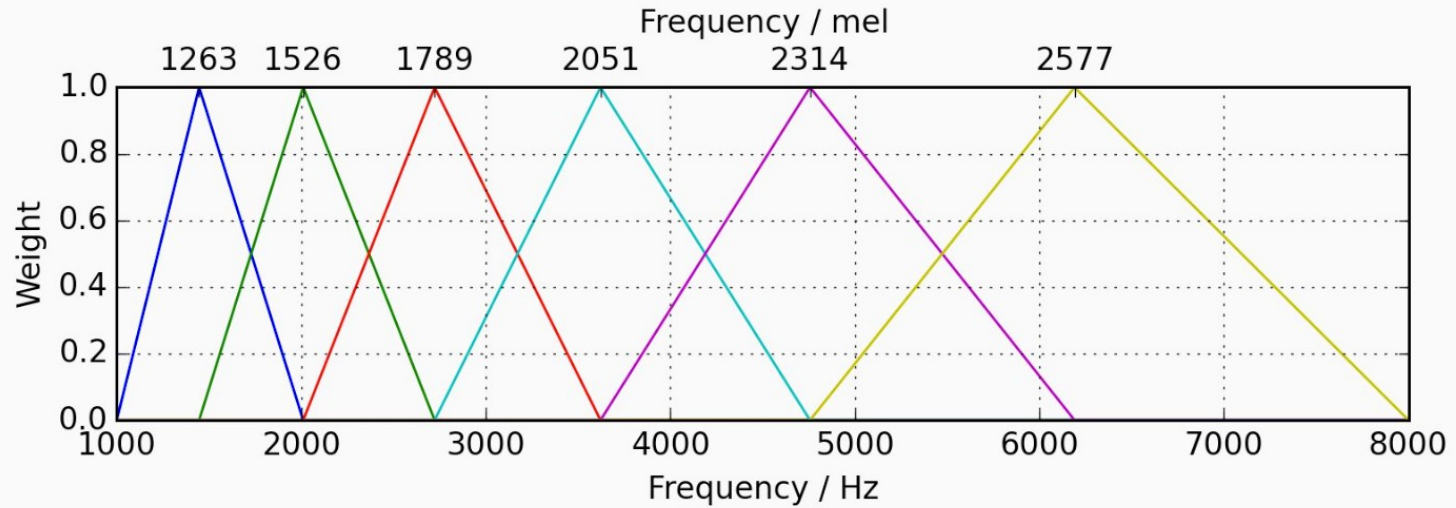
Data Preprocessing: Spectrograms



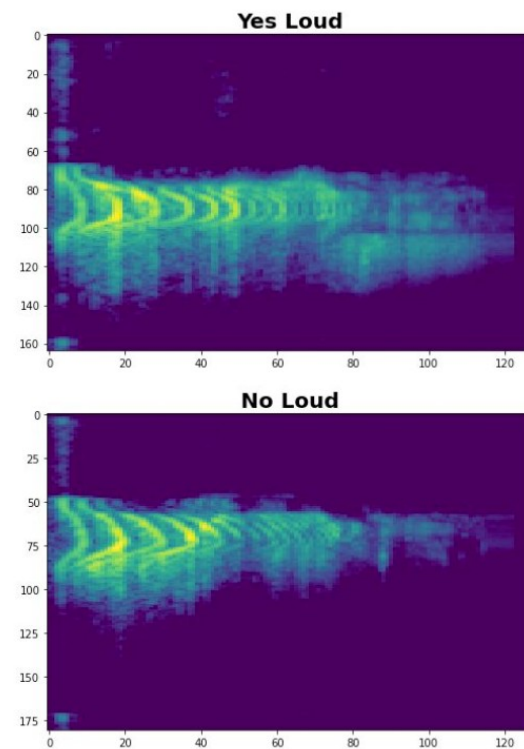
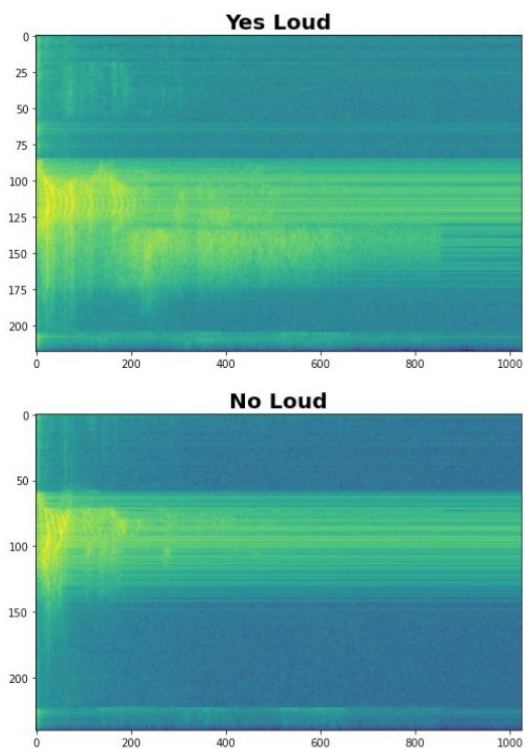


The [lower band frequencies](#) is much more crisper to us

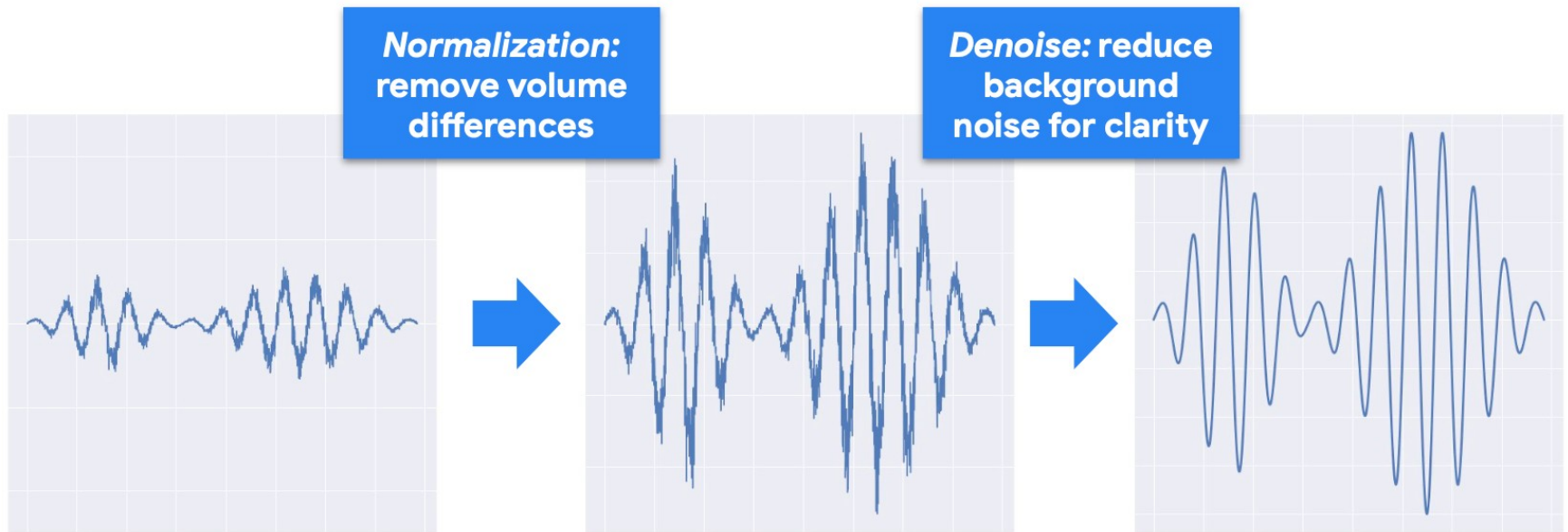
Mel Filterbanks



Spectrograms v. MFCCs



Additional Feature Engineering



EdgeImpulse

- Goal:
 - develop a system that can blink an LED when it detects the word Yes (or Hai or Naeam or Si or Oui or...)



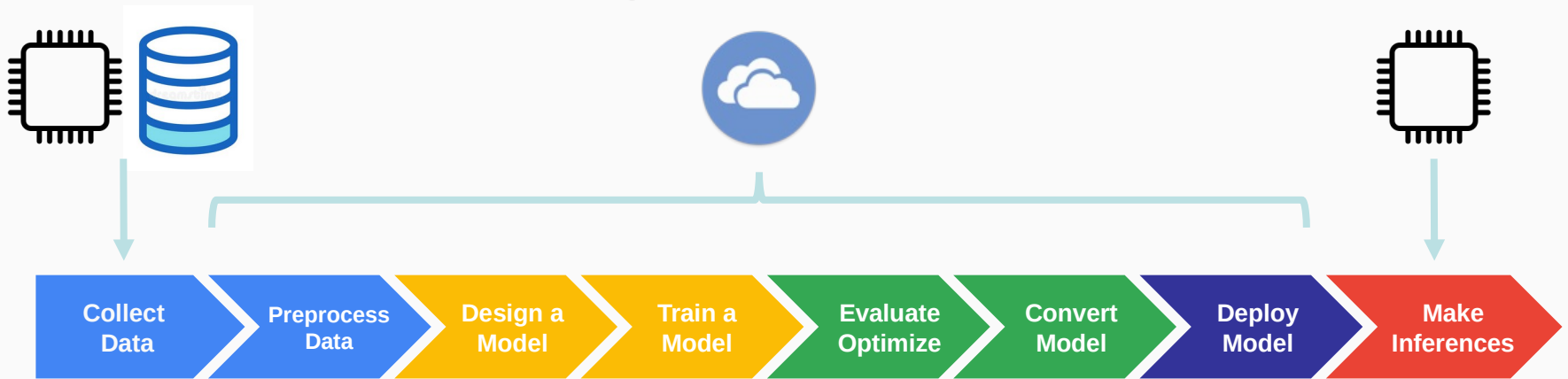
Machine Learning Workflow



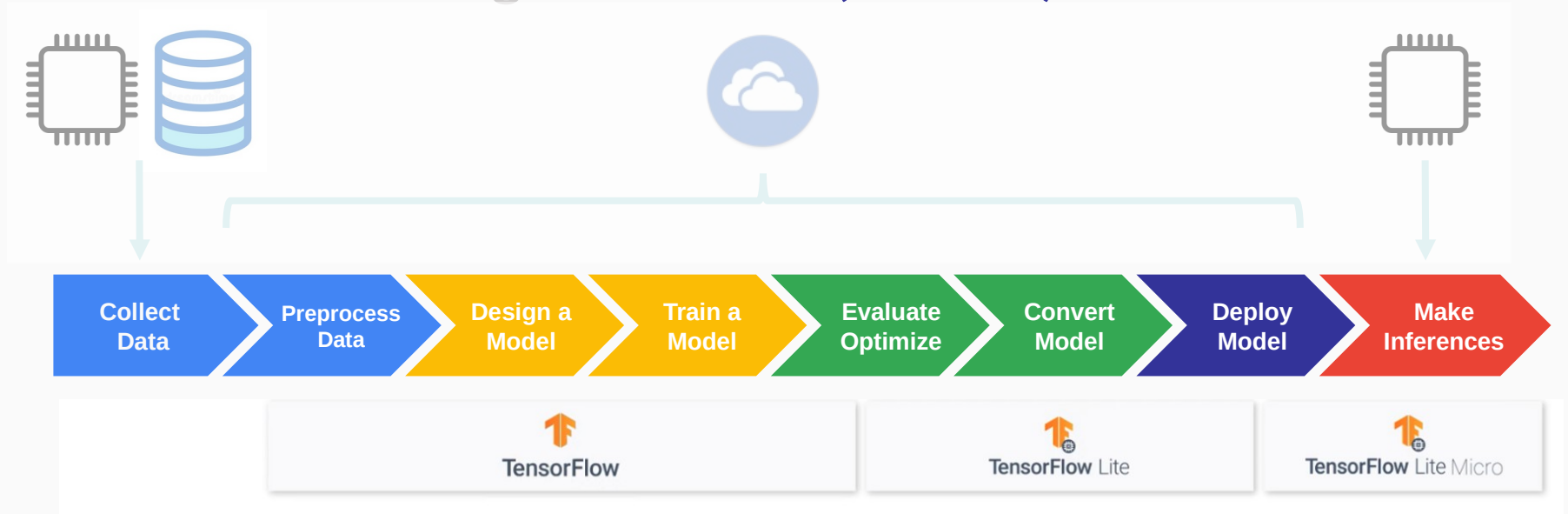
Tiny Machine Learning Workflow (“What”)



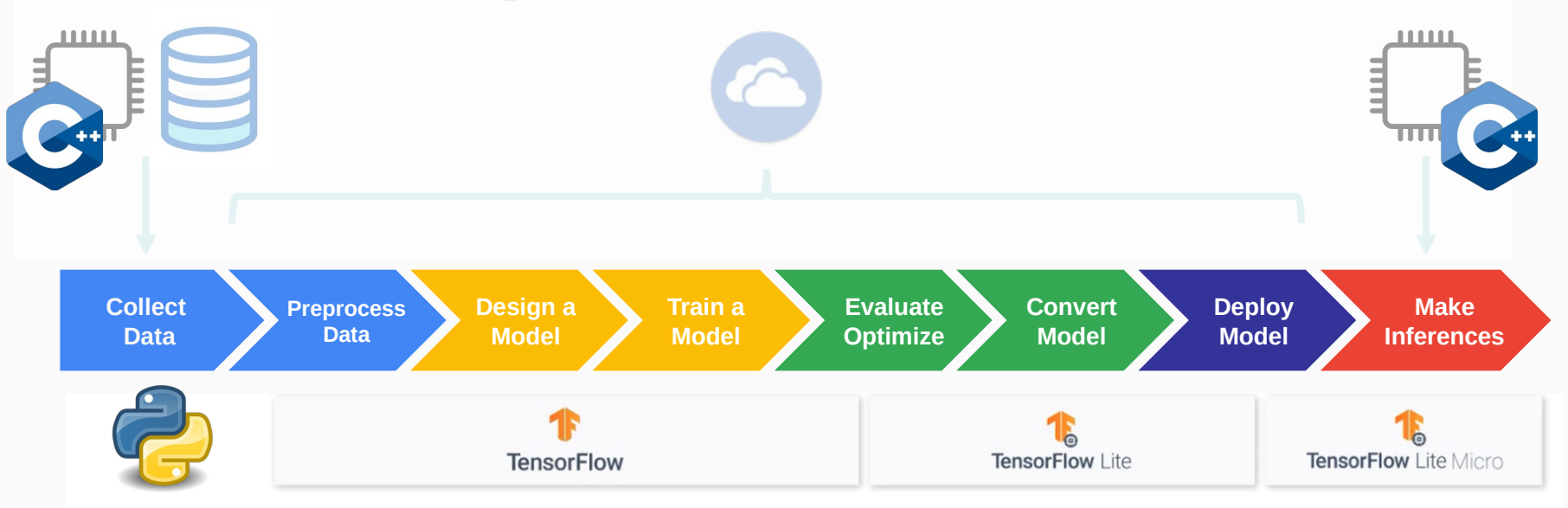
Tiny Machine Learning Workflow (“Where”)



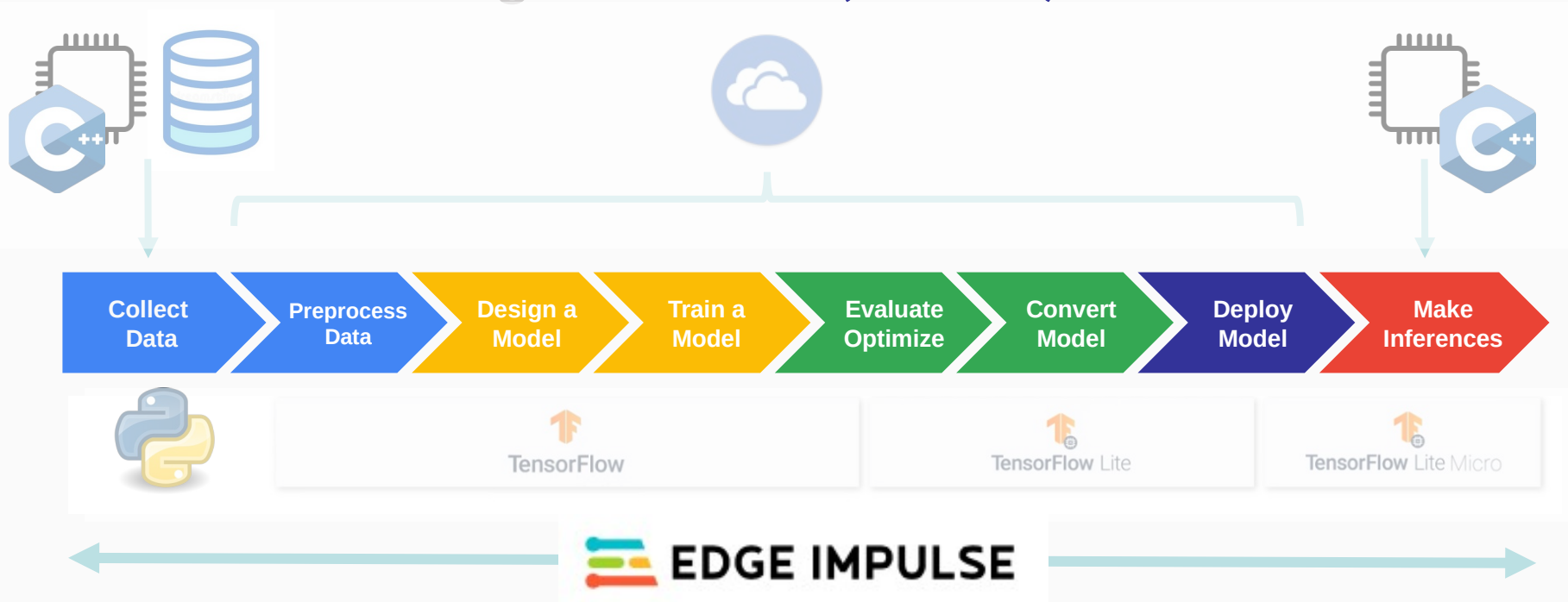
Machine Learning Workflow (“How”)

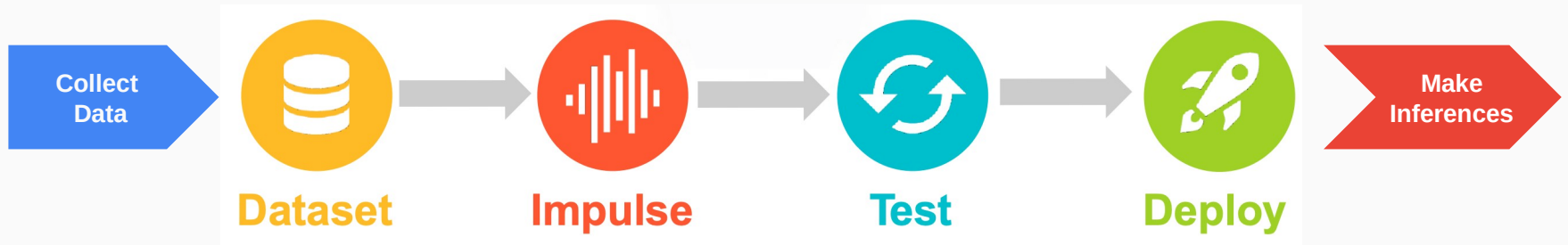


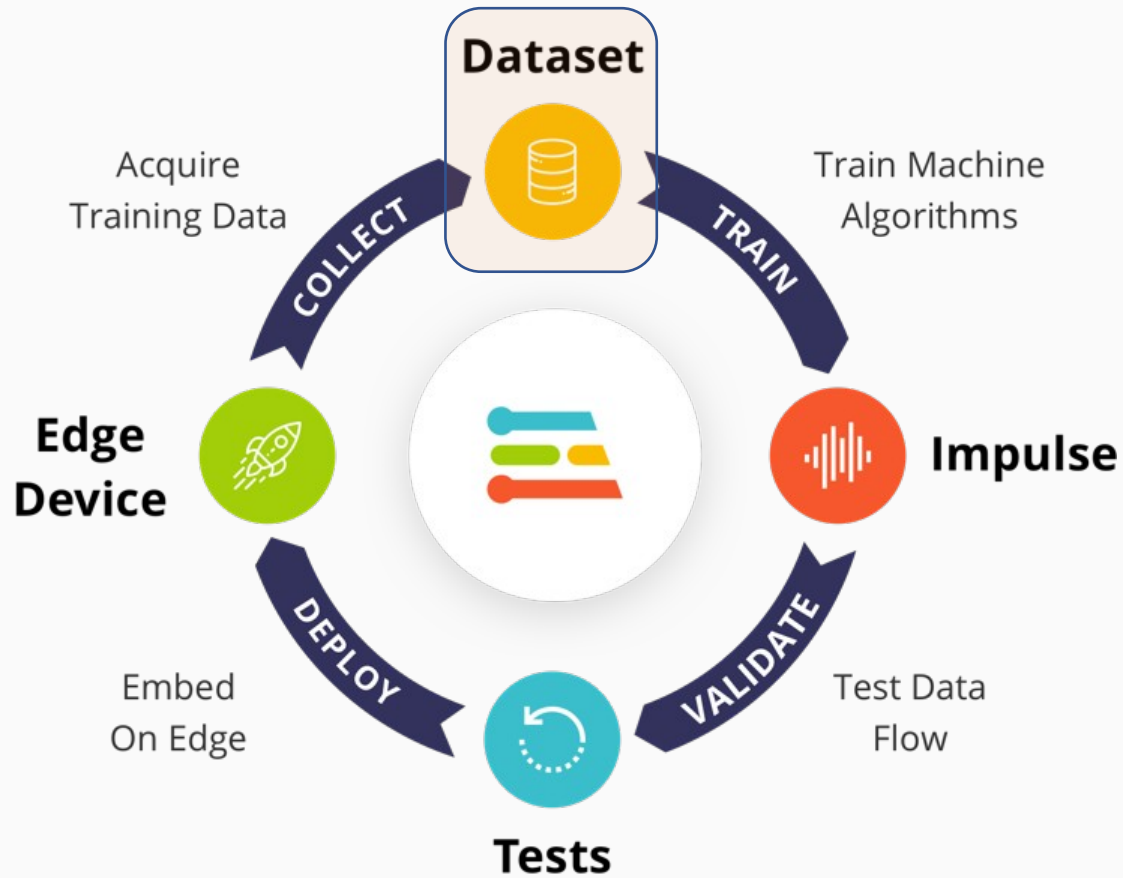
Machine Learning Workflow (“How”)



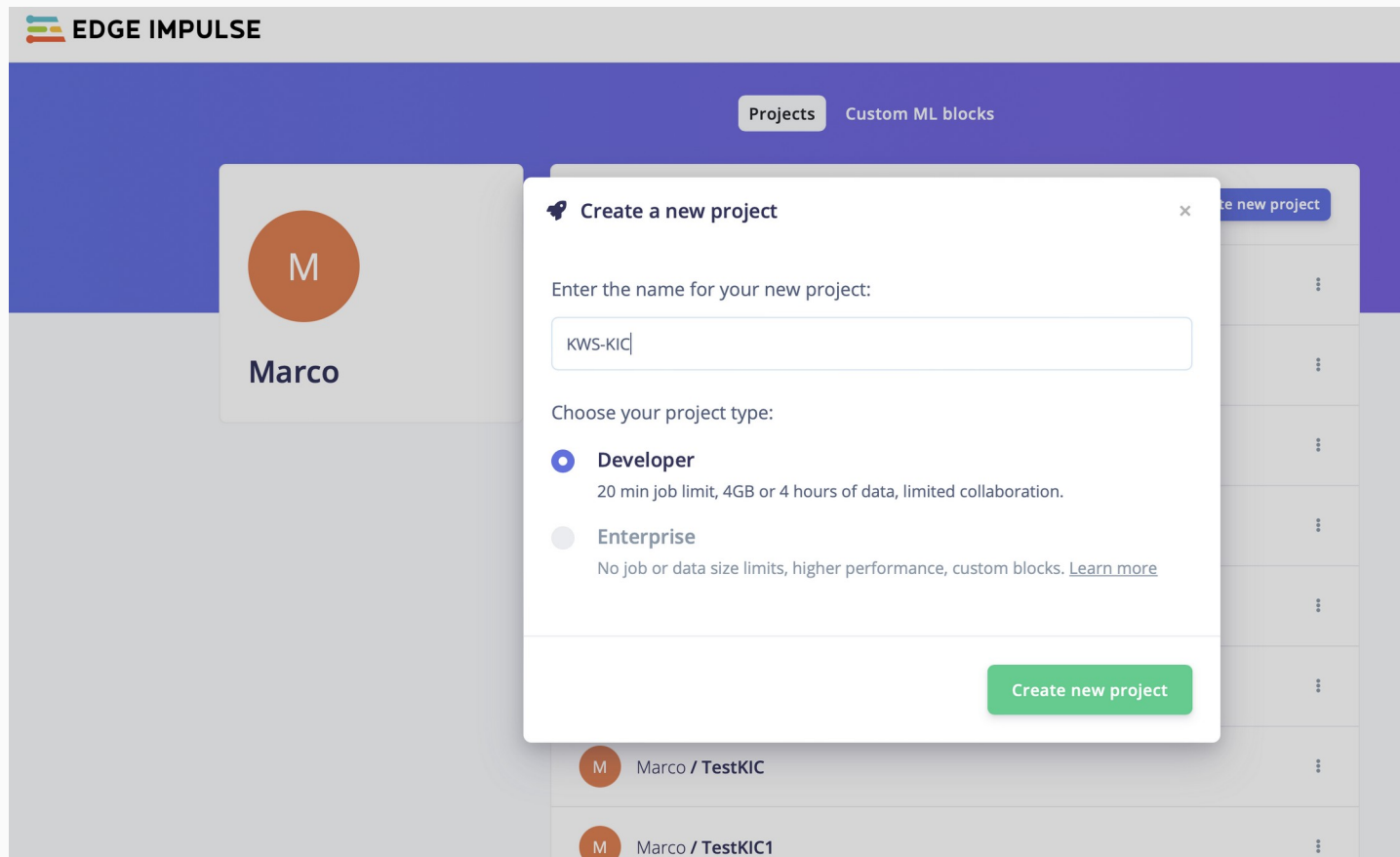
Machine Learning Workflow (“How”)



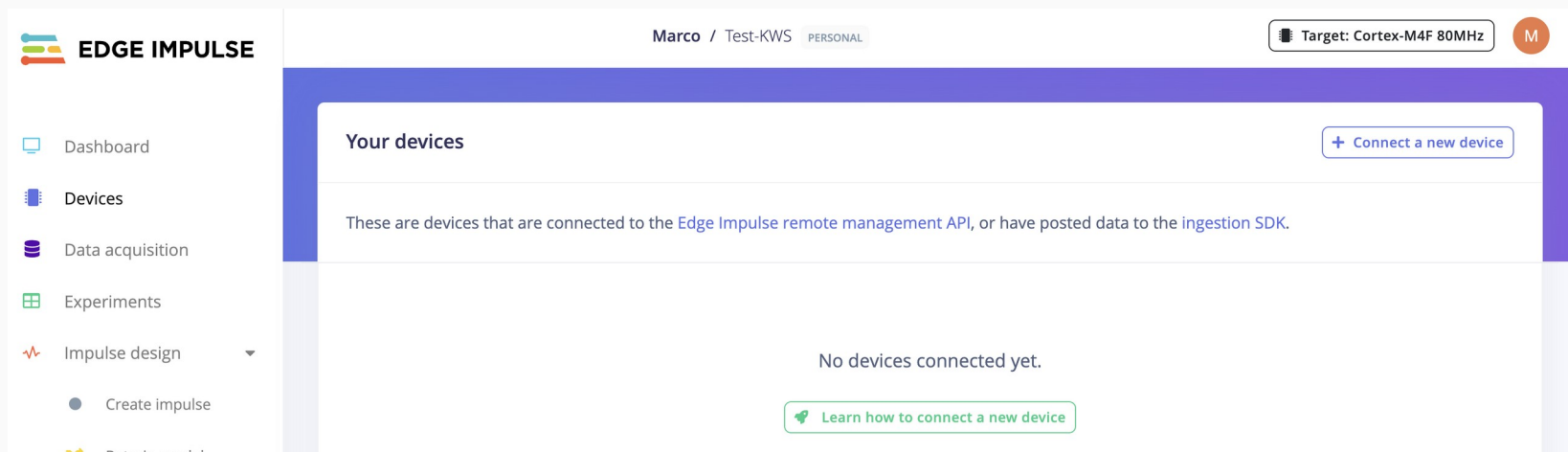




- Register a new account in Edge Impulse. Create a **new project** and give it a name:



- Once the project is created, click on **Devices** on the left to add your smartphone as input device. You could also upload data from an existing dataset or add a sensor.



Merge / Test KWS

Collect new data

Collect data directly from your phone, computer, device, or development board.



Scan QR code to connect to your phone



Connect to your computer



Connect your device or development board

Collect new data



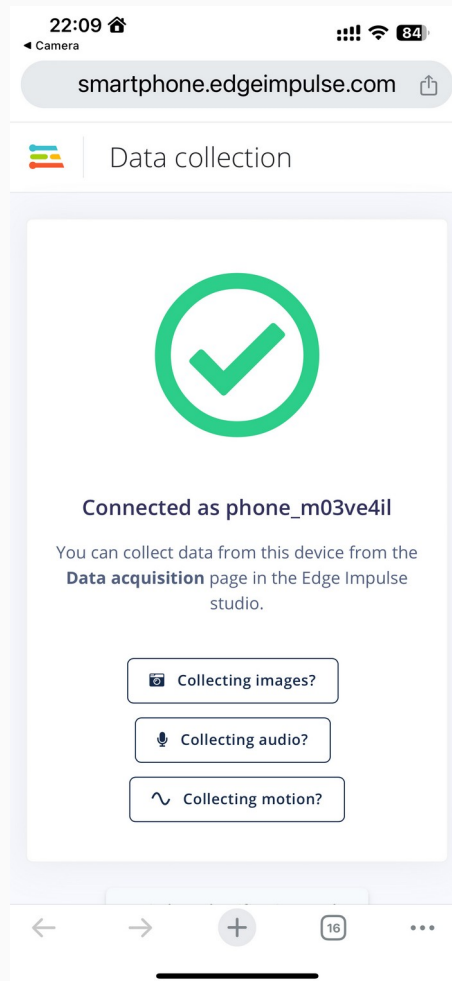
Device "phone_m03ve4il" is now connected

Go to 'Data acquisition' to collect data from this device.



Get started!

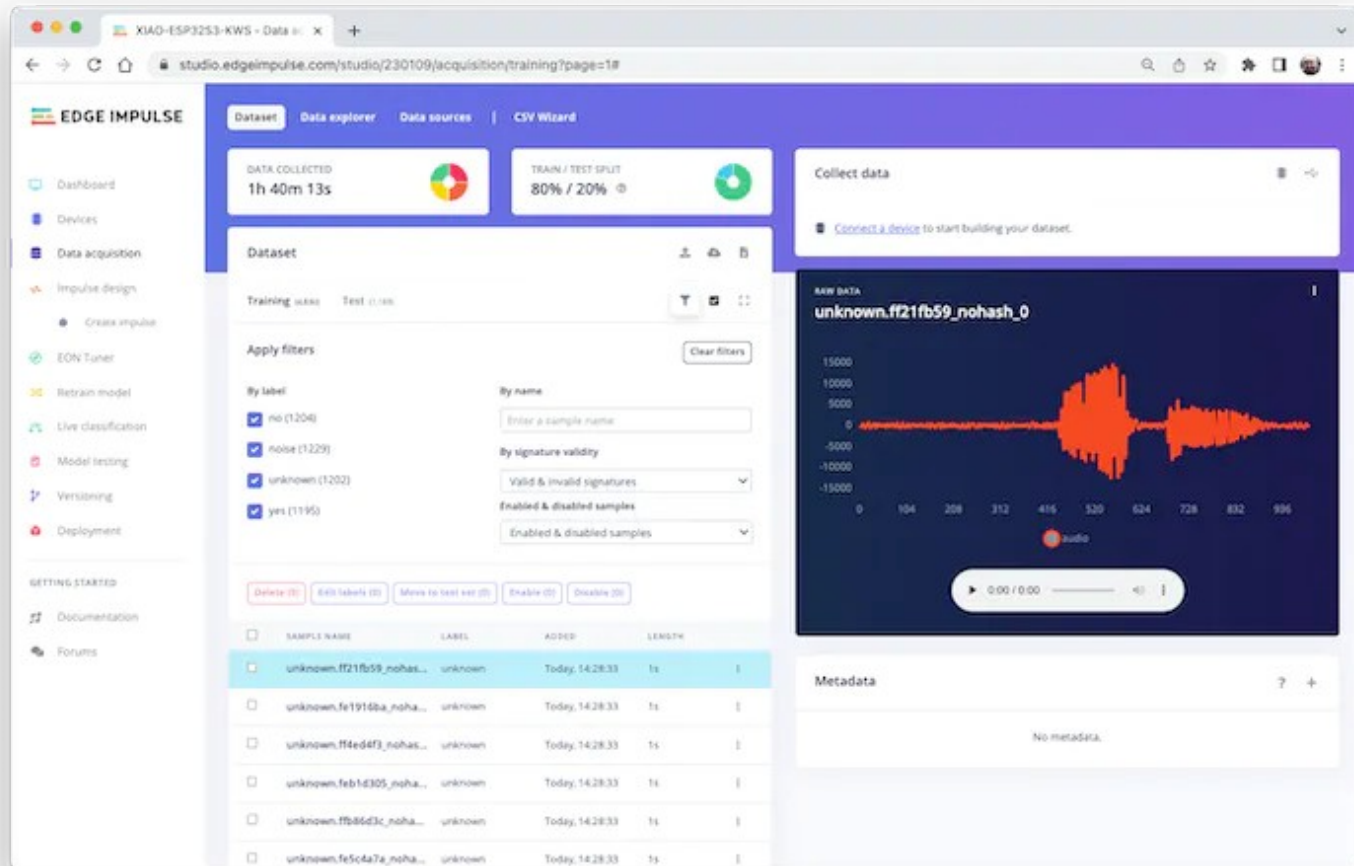
[< Back](#)



Split data

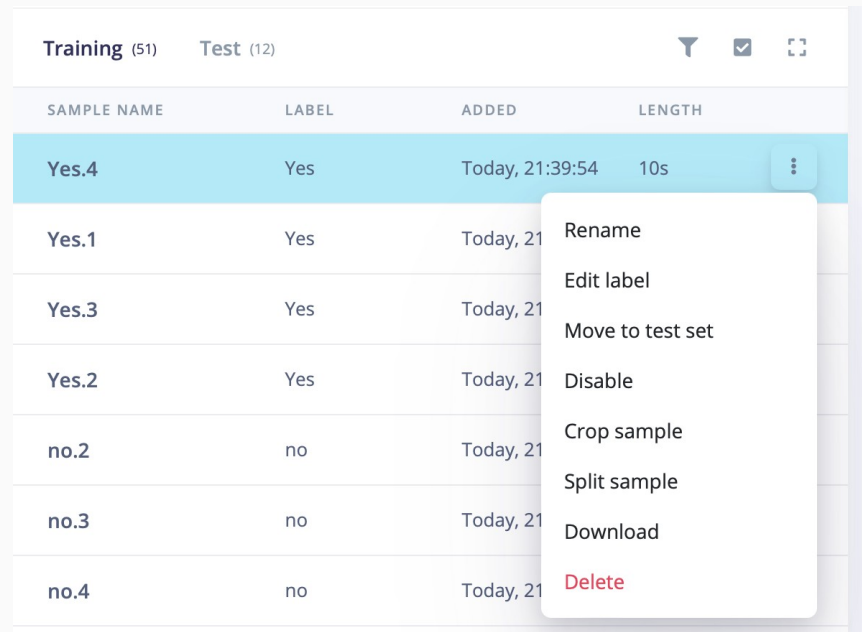
- To detect a machine learning model behavior, we need to use observations that aren't used in the training process. Otherwise, the evaluation of the model would be biased.
- Using the training observations for model evaluation is like giving a class of students a set of questions, then asking some of the questions in the final exam. We can't know whether the students really learned the subject or just memorized some specific data.
- The simplest method is to divide the whole dataset into two sets. Then use one for training and the other for model evaluation.
- Empirical studies show that the best results are obtained if we use **20-30% of the data for testing**, and the remaining **70-80% of the data for training**.

- The samples will now appear in the Data acquisition section.



Lab

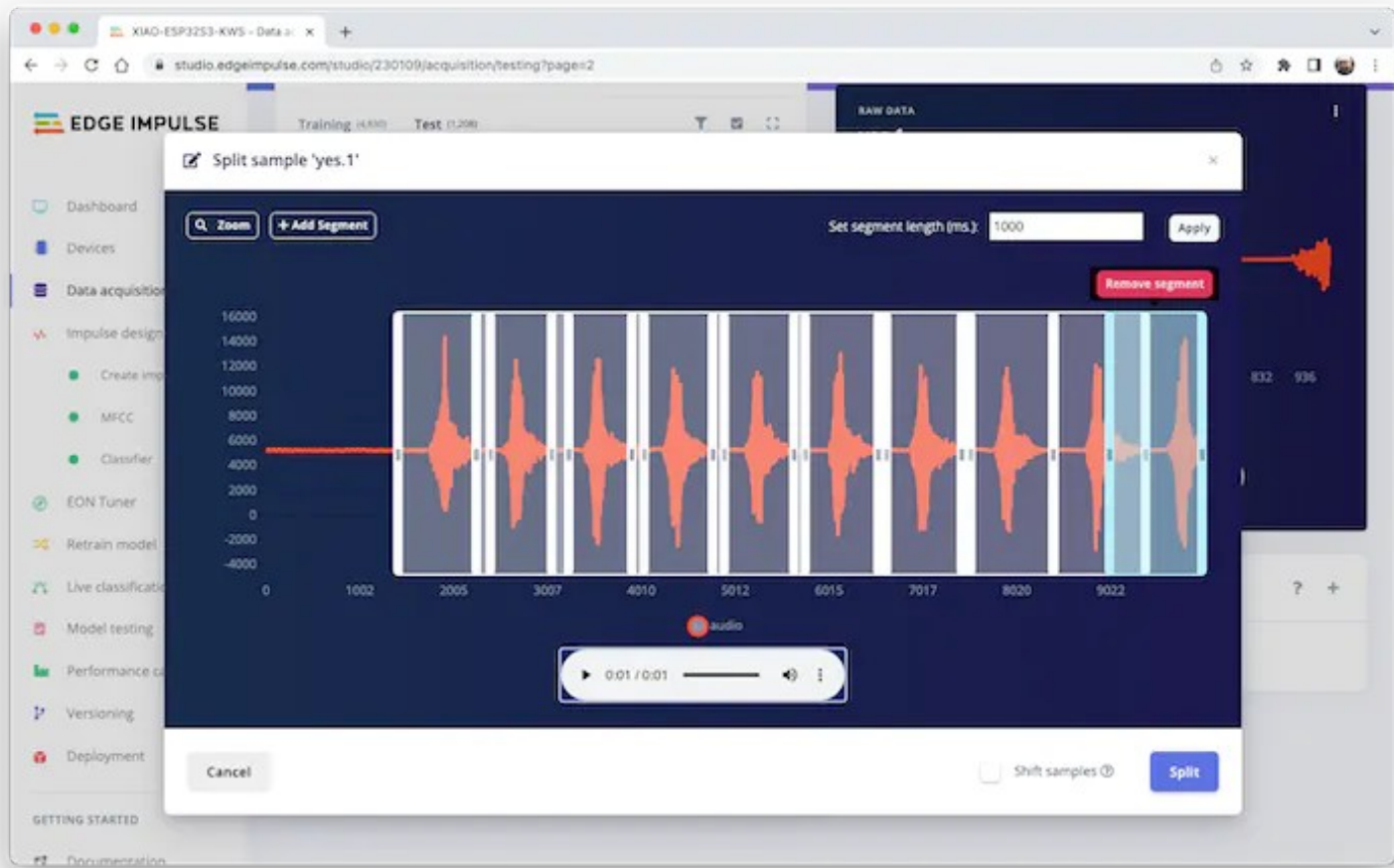
- All data from the Edge Impulse dataset have a 1 second length. We might want to split the 1 second samples into smaller chunks.
- Click on three dots after the sample name and select **Split sample**.



The screenshot shows the Edge Impulse dataset interface. At the top, there are tabs for 'Training (51)' and 'Test (12)'. Below the tabs is a table with columns: 'SAMPLE NAME', 'LABEL', 'ADDED', and 'LENGTH'. The first row, 'Yes.4', is highlighted in light blue. A context menu is open for this row, showing options: 'Rename', 'Edit label', 'Move to test set', 'Disable', 'Crop sample', 'Split sample', 'Download', and 'Delete' (in red). The table contains several other rows with labels 'Yes' and 'no'.

SAMPLE NAME	LABEL	ADDED	LENGTH
Yes.4	Yes	Today, 21:39:54	10s
Yes.1	Yes	Today, 21:39:54	10s
Yes.3	Yes	Today, 21:39:54	10s
Yes.2	Yes	Today, 21:39:54	10s
no.2	no	Today, 21:39:54	10s
no.3	no	Today, 21:39:54	10s
no.4	no	Today, 21:39:54	10s

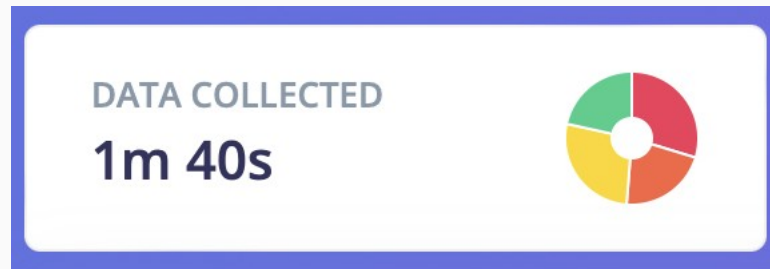
- Once inside the tool, split the dat. If necessary, add or remove segments:



- This procedure should be repeated for all samples.
- Note: For longer audio files (minutes), first, split into 10-second segments and after that, use the tool again to get the final 1-second splits.



It is important for the dataset to be balanced!

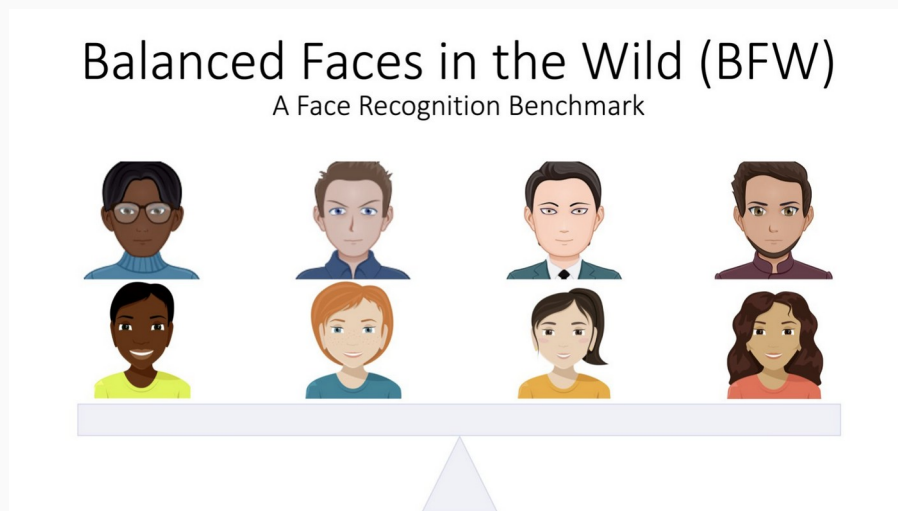


Balanced datasets

- Classification models attempt to categorize data into different buckets. In an imbalanced dataset, one bucket makes up a large portion of the training dataset (the majority class), while the other bucket is underrepresented in the dataset (the minority class). The problem with a model trained on imbalanced data is that the model learns that it can achieve high accuracy by consistently predicting the majority class, even if recognizing the minority class is equal or more important when applying the model to a real-world scenario.
- Consider the case of collecting training data for a model predicting medical conditions. Most of the patient data collected, let's say 95 percent, will likely fall into the healthy bucket, while the sick patients make up a much smaller portion of the data. During training, the classification model learns that it can achieve 95 percent accuracy if it predicts “healthy” for every piece of data it encounters. That's a huge problem because what doctors really want the model to do is identify those patients suffering from a medical condition.

Balanced datasets

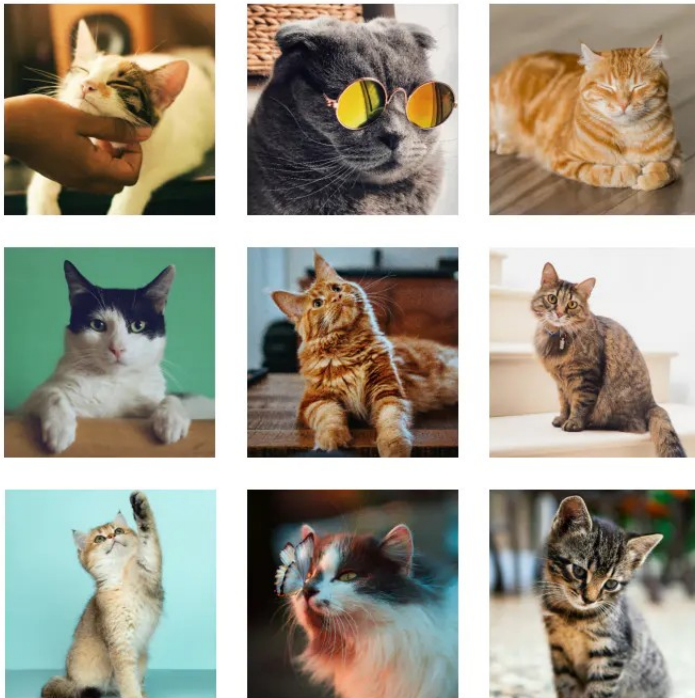
- **Imbalanced datasets** create challenges for predictive modeling, but they're a common and anticipated problem because the real world is full of imbalanced examples.



- **Balancing a dataset** makes training a model easier because it helps prevent the model from becoming biased towards one class. In other words, the model will no longer favor the majority class just because it contains more data.

Unbalanced Dataset

Cats



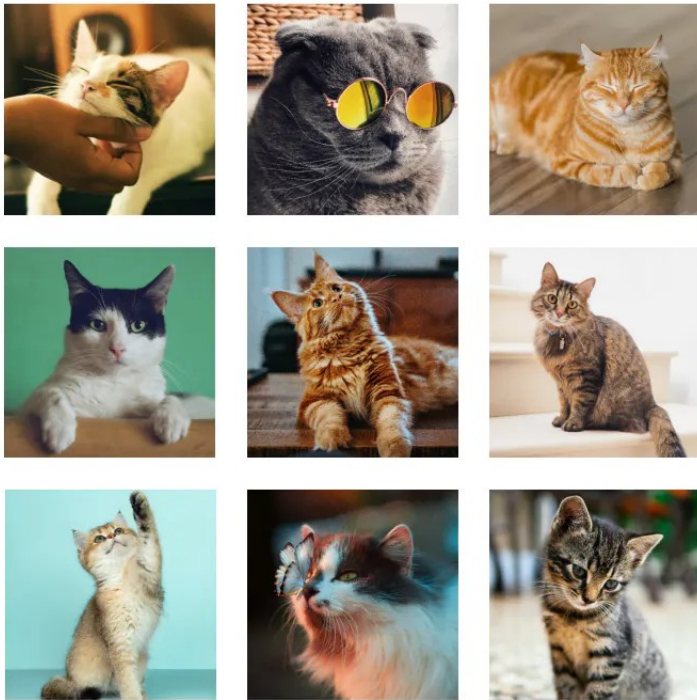
Dogs



Balanced datasets

Get More Data

Cats

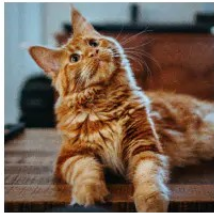


Dogs



Undersampled

Cats



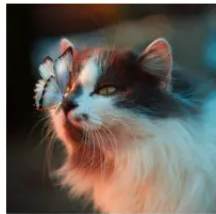
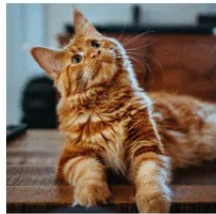
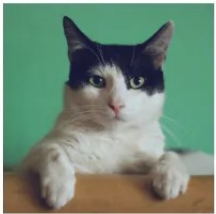
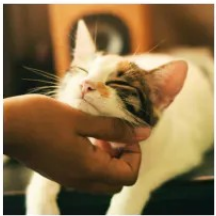
Dogs



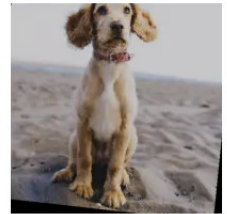
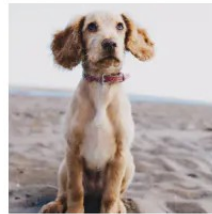
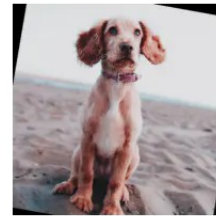
Balanced datasets

Oversampled

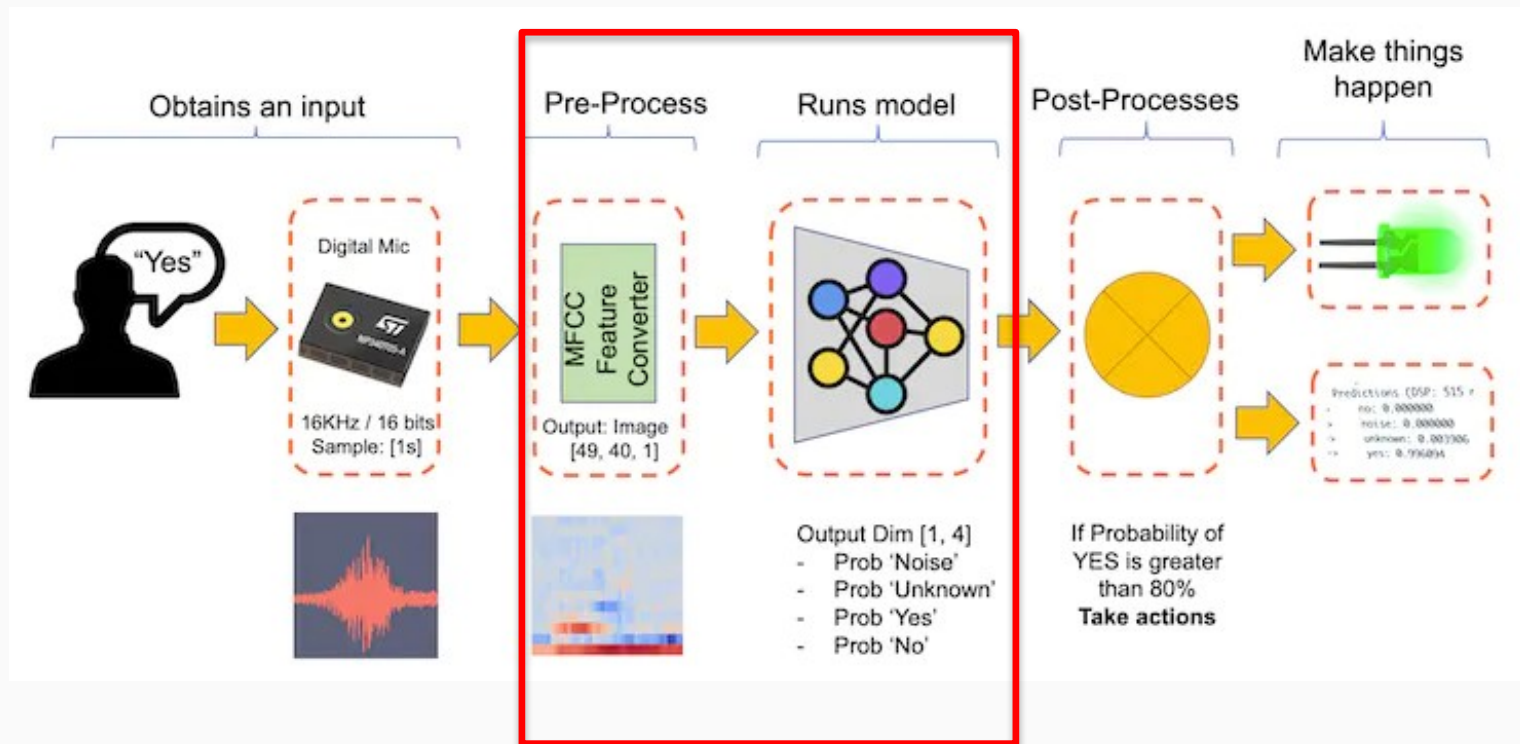
Cats



Dogs



- We will now be creating an Impulse (Pre-Process / Model definition)



- An impulse takes raw data, uses signal processing to extract features, and then uses a learning block to classify new data.

- Dashboard
- Devices
- Data acquisition
- Impulse design**
 - Create impulse
- EON Tuner
- Retrain model
- Live classification
- Model testing
- Versioning
- Deployment

Dataset | Data explorer | Data sources | CSV Wizard

DATA COLLECTED

1m 40s

TRAIN / TEST SPLIT

88% / 12%

Dataset

Training (89) Test (12)

SAMPLE NAME	LABEL	ADDED	LENGTH
noise.dude_miaowi...	noise	Today, 21:38:42	1s
noise.dude_miaowi...	noise	Today, 21:38:42	1s
noise.dude_miaowi...	noise	Today, 21:38:42	1s
noise.doing_the_dis...	noise	Today, 21:38:42	1s
noise.exercise_bike....	noise	Today, 21:38:42	1s
noise.doing_the_dis...	noise	Today, 21:38:42	1s
noise.exercise_bike....	noise	Today, 21:38:42	1s

RAW DA

no.1

30000

25000

20000

15000

10000

5000

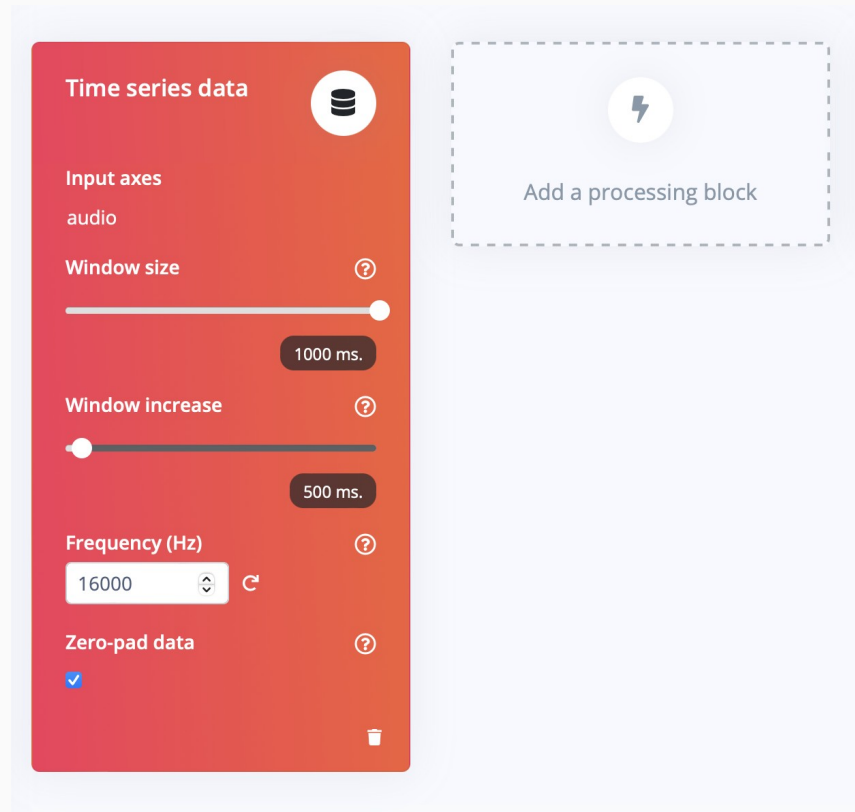
0

-5000

-10000

-15000

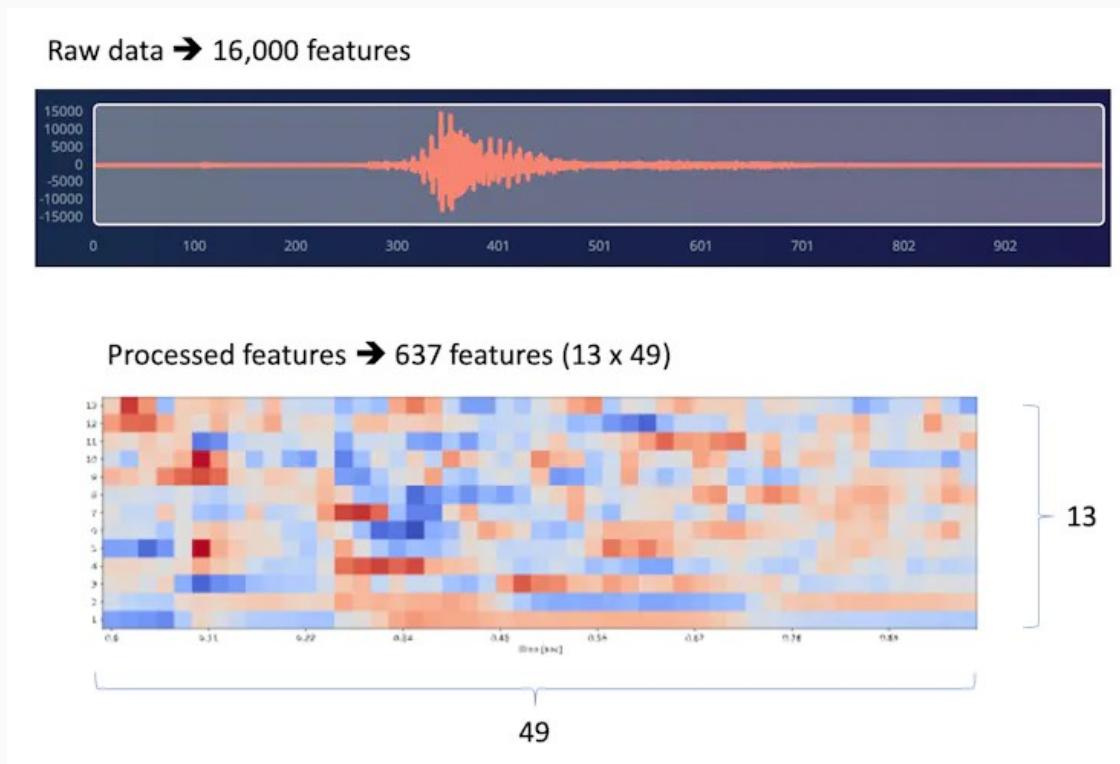
- First, we will take the data points with a 1-second window, augmenting the data, sliding that window each 500ms. Note that the option zero-pad data is set. It is important to fill with zeros samples smaller than 1 second (in some cases, you can reduce the 1000 ms window on the split tool to avoid noises and spikes).



- In the Add a processing block, select MFCC.

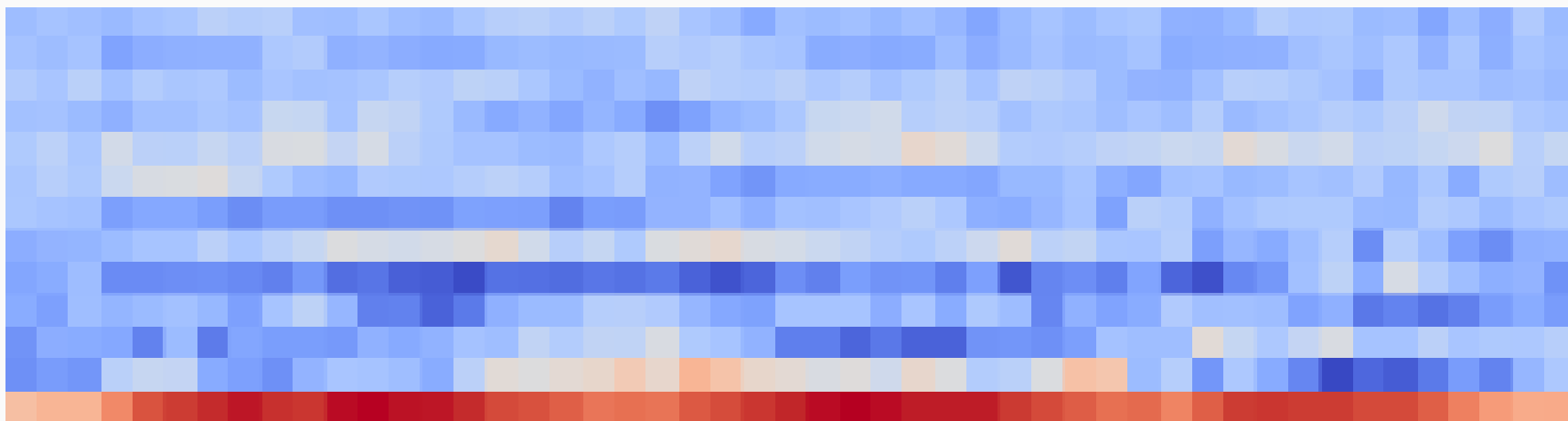
DESCRIPTION	AUTHOR	RECOMMENDED
Audio (MFCC) Extracts features from audio signals using Mel Frequency Cepstral Coefficients, great for human voice.	Edge Impulse 	Add
Audio (MFE) Extracts a spectrogram from audio signals using Mel-filterbank energy features, great for non-voice audio.	Edge Impulse 	Add
Spectrogram Extracts a spectrogram from audio or sensor data, great for non-voice audio or data with continuous frequencies.	Edge Impulse	Add
Audio (Syntiant) Syntiant only. Compute log Mel-filterbank energy features from an audio signal.	Syntiant	Add
Raw Data Use data without pre-processing. Useful if you want to use deep learning to learn features.	Edge Impulse	Add

- Each 1-second audio sample will be pre-processed and converted to an image (for example, $13 \times 49 \times 1$). MFCC extracts features from audio signals using Mel Frequency Cepstral Coefficients, which are great for the human voice.



- **Mel Frequency Cepstral Coefficients (MFCCs)** is a way of extracting features from an audio. The MFCC uses the MEL scale to divide the frequency band to sub-bands and then extracts the Cepstral Coefficients using Discrete Cosine Transform (DCT). MEL scale is based on the way humans distinguish between frequencies which makes it very convenient to process sounds.
- Humans can hear roughly between 20Hz to 20KHz. The perception of sound is non-linear, and you can better distinguish between low frequency sounds than high frequency sounds e.g., humans can clearly hear the difference between 100Hz and 200Hz but not between 15kHz and 15.1kHz.

- An **MFCC spectrogram** is a specially tuned spectrogram which highlights frequencies which are common in human speech.
- In the spectrogram **the vertical axis represents the frequencies**, and the **horizontal axis represents time**. The patterns visible in a spectrogram contain information about what type of sound it represents. For example, the spectrogram in this image shows "Hello world":



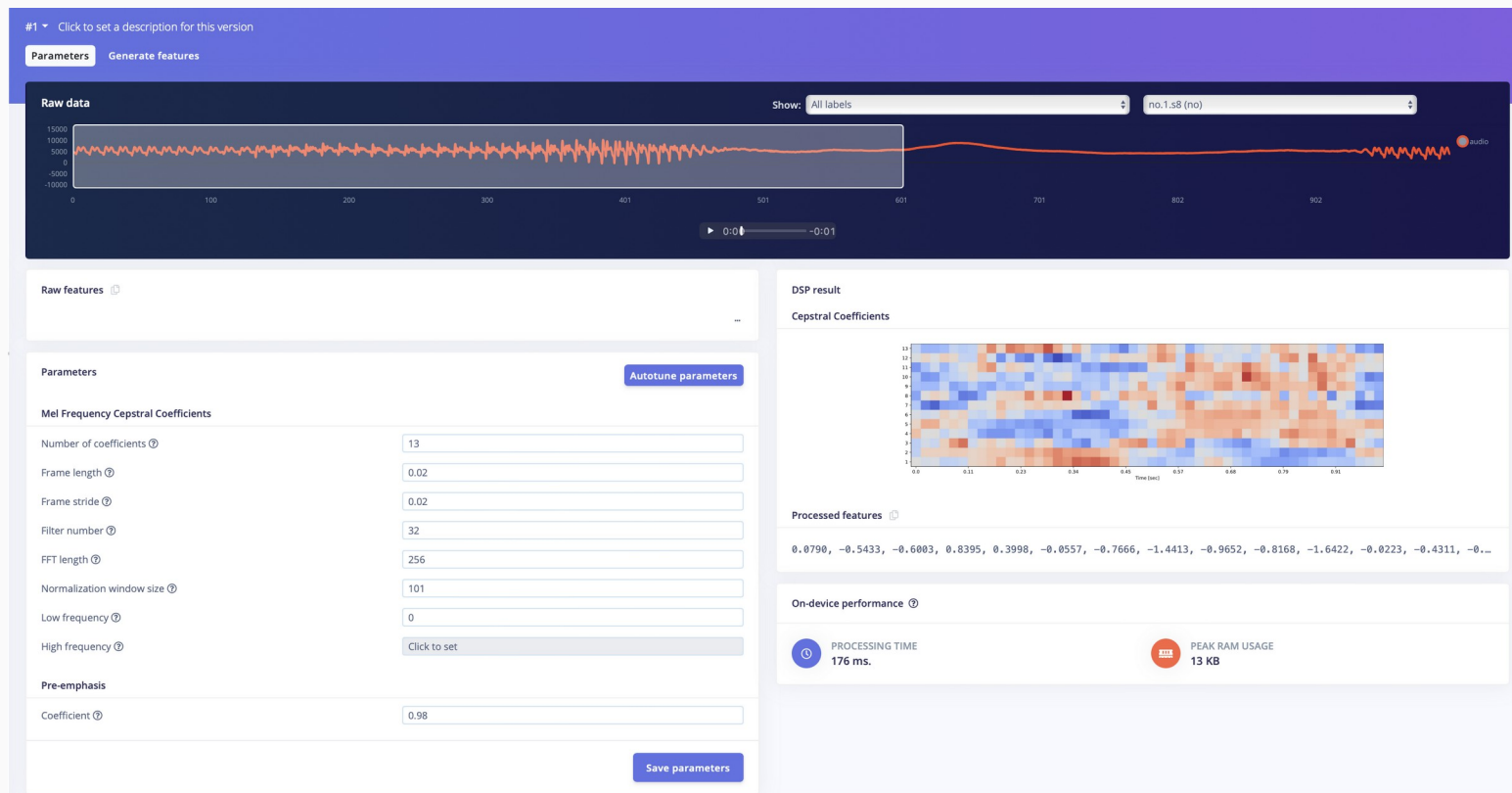
- In the Add a learning block, select Classification. You can now save the Impulse.

The screenshot displays the 'Add a learning block' interface with four blocks arranged horizontally. Below the 'Audio (MFCC)' and 'Classification' blocks are dashed boxes containing their respective icons (a lightning bolt and a flask).

- Time series data** (red block, database icon):
 - Input axes: audio
 - Window size: 1000 ms.
 - Window increase: 500 ms.
 - Frequency (Hz): 16000
 - Zero-pad data: ☒
- Audio (MFCC)** (white block, lightning bolt icon):
 - Name: MFCC
 - Input axes (1): ☒ audio
- Classification** (purple block, flask icon):
 - Name: Classifier
 - Input features: ☒ MFCC
 - Output features: 4 (Yes, no, noise, unknown)
- Output features** (green block, checkmark icon):
 - 4 (Yes, no, noise, unknown)

A green **Save Impulse** button is located below the 'Output features' block.

- If you click on MFCC on the left side of the window, you should obtain the following information. Leave the default values and click on **Save Parameters**.



- We will now generate the features. Click on Generate features.

#1 ▾ Click to set a description for this version

Parameters **Generate features**

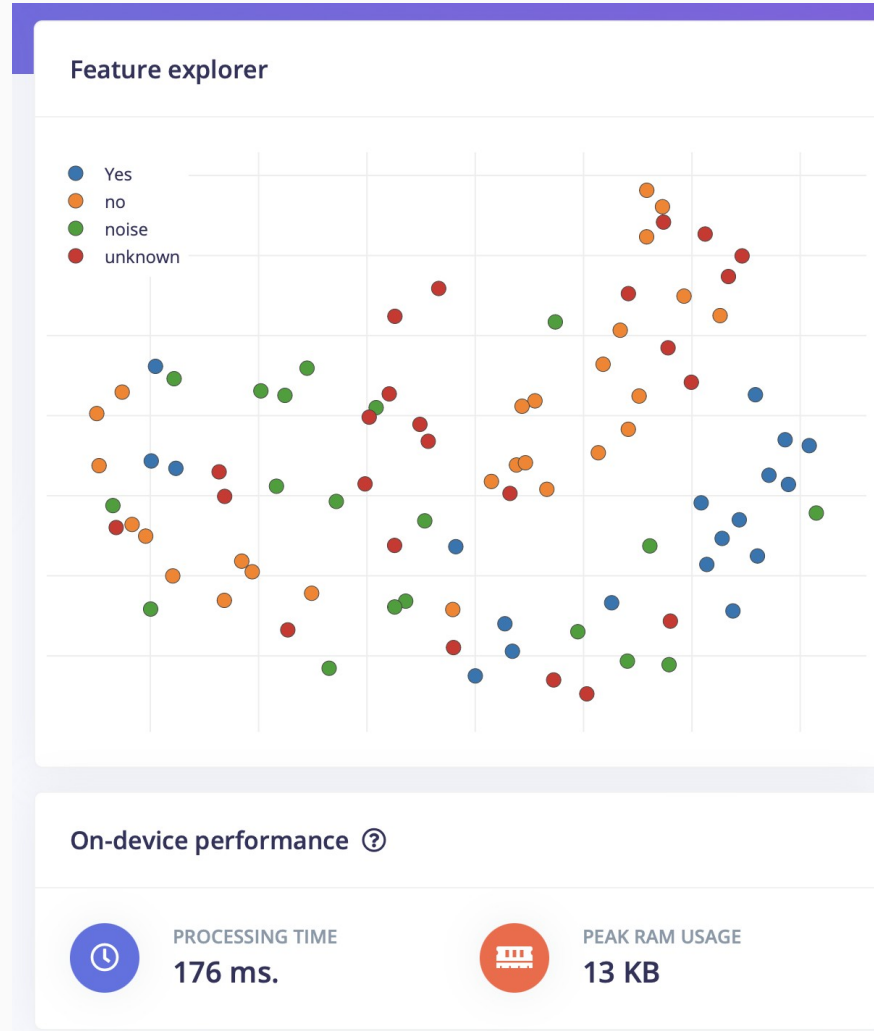
Training set

Data in training set	1m 28s
Classes	4 (Yes, no, noise, unknown)
Training windows	88

Generate features

© 2023 [EdgeImpulse Inc.](#) All rights reserved

- The Feature explorer should be organized in "clusters" of separated features. With our dataset the features are "mixed" and this is not a good sign.
- The on-device performance part shows the processing time for inference and the peak ram usage. Both are low!



- We are finally ready for Classification! Select Classifier on the left side of the window.

The screenshot displays the Edge Impulse web interface. On the left sidebar, the 'Classifier' option is highlighted with a red rectangle. The main panel shows the 'Neural Network settings' section, which includes 'Training settings' (Number of training cycles: 100, Learning rate: 0.005) and 'Advanced training settings'. Below this is the 'Audio training options' section with a 'Data augmentation' checkbox. The 'Neural network architecture' section shows 'Architecture presets' with '1D Convolutional (Default)' and '2D Convolutional' options. At the bottom, a blue bar indicates the 'Input layer (650 features)'.

EDGE IMPULSE

Marco / KWS-KIC

#1 ▾ Click to set a description for this version

Neural Network settings

Training settings

Number of training cycles ⓘ 100

Learning rate ⓘ 0.005

Advanced training settings ▾

Audio training options

Data augmentation ⓘ ☐

Neural network architecture

Architecture presets ⓘ 1D Convolutional (Default) 2D Convolutional

Input layer (650 features)

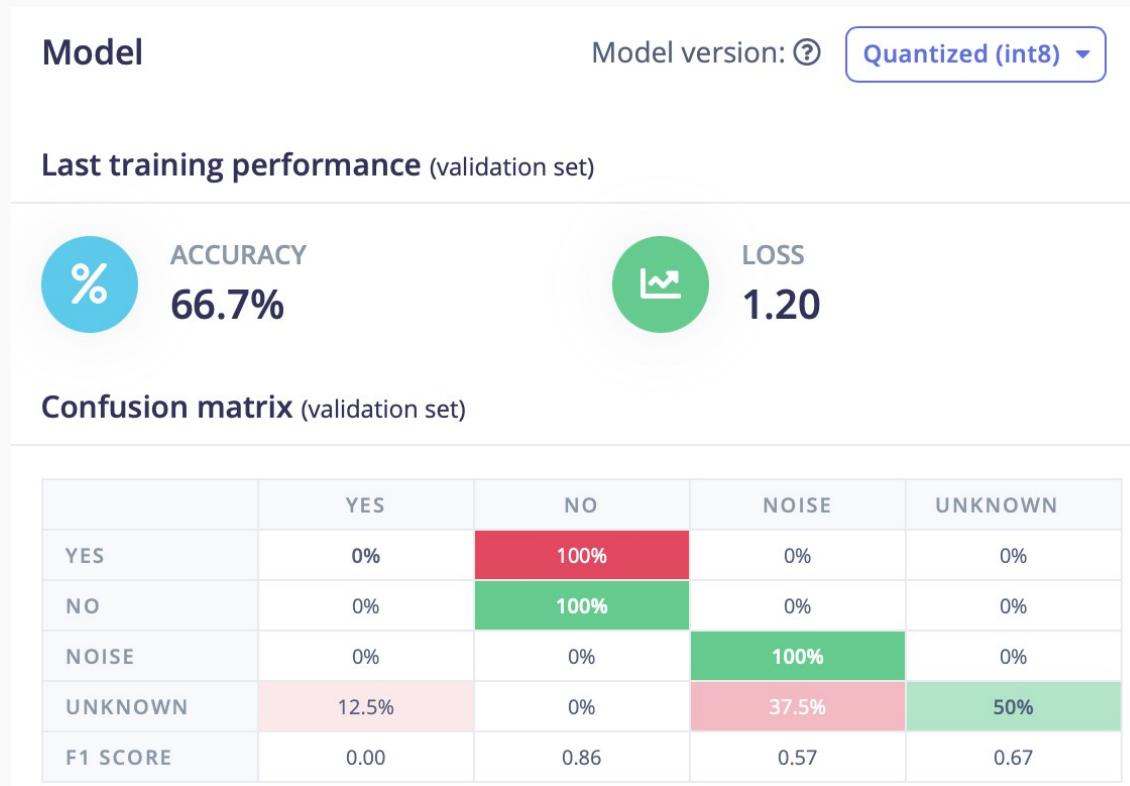
Neural Network

- It's time to start training a **neural network**. Neural networks are algorithms that can learn to recognize patterns that appear in their training data. The network that we're training here will take the MFCC as an input and try to map this to one of three classes—Yes, No, Noise or Unknown.
- A neural network is composed of layers of virtual "neurons. An input—in our case, an MFCC spectrogram—is fed into the first layer of neurons, which filters and transforms it based on each neuron's unique internal state. The first layer's output is then fed into the second layer, and so on, gradually transforming the original input into something radically different. In this case, the spectrogram input is transformed over four intermediate layers into just two numbers: the probability that the input represents your keyword, and the probability that the input represents 'noise' or 'unknown'.

Neural Network

- During training, the internal state of the neurons is gradually tweaked and refined so that the network transforms its input in just the right ways to produce the correct output. This is done by feeding in a sample of training data, checking how far the network's output is from the correct answer, and **adjusting the neurons' internal state to make it more likely that a correct answer is produced next time**. When done thousands of times, this results in a trained network.
- In our case, the basic architecture is defined with two blocks of Conv1D + MaxPooling (with 8 and 16 neurons, respectively) and a 0.25 Dropout. And on the last layer, after Flattening four neurons, one for each class.
- As hyper-parameters, we will have a Learning Rate of 0.005 and a model that will be trained by 100 epochs.
- **Please select Data Augmentation.**

- The result is not great. Testing the model with the data put apart before training (Test Data), we got an accuracy of approximately 67%. → more data is needed!



- Before deployment of the model on the Xiao device, it is possible to perform Live Classification using a smartphone. Go to the **Live Classification** section and click on **Connect a Development board**:

Marco / KWS-KIC

Connect a development board

Classify new data

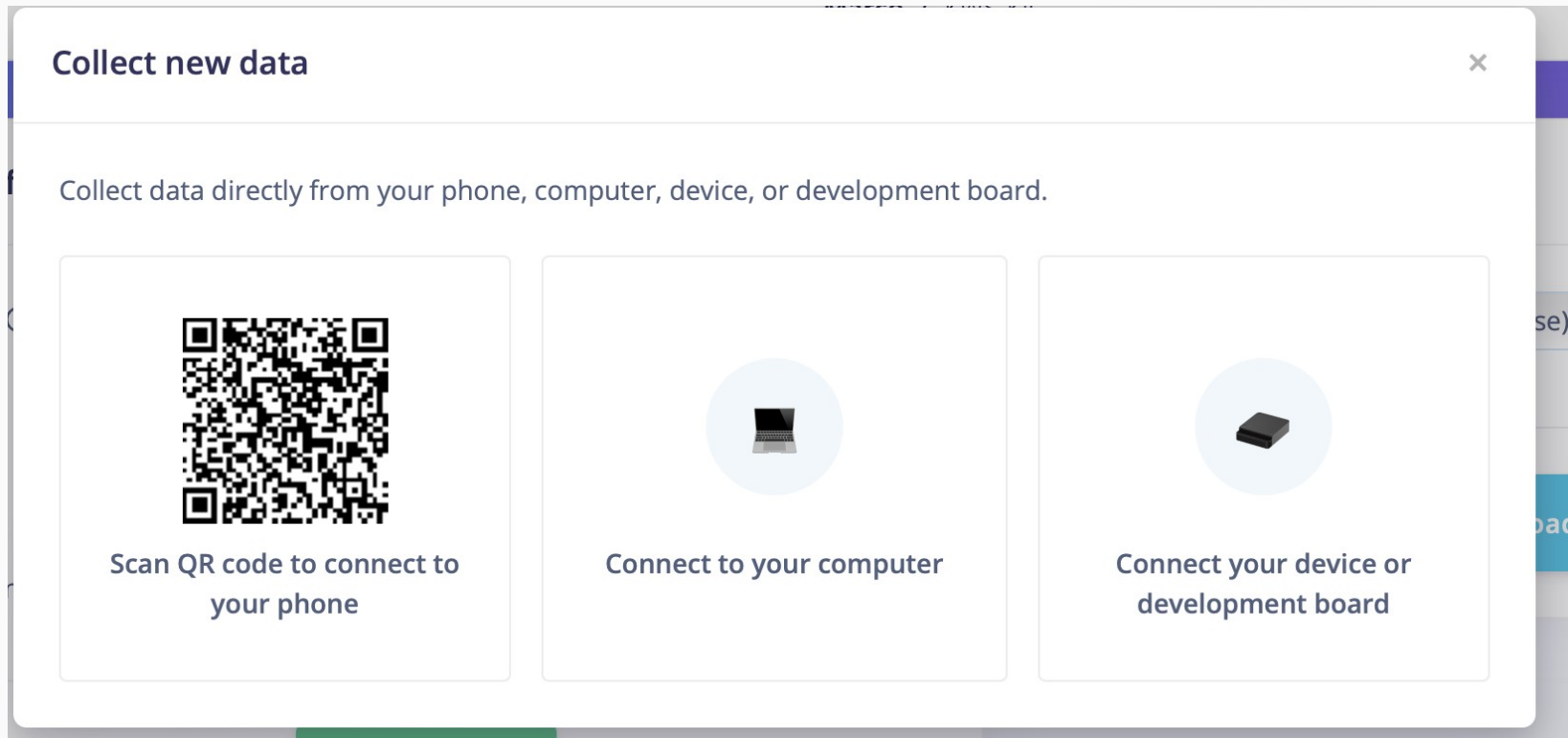
Device ?

Sensor

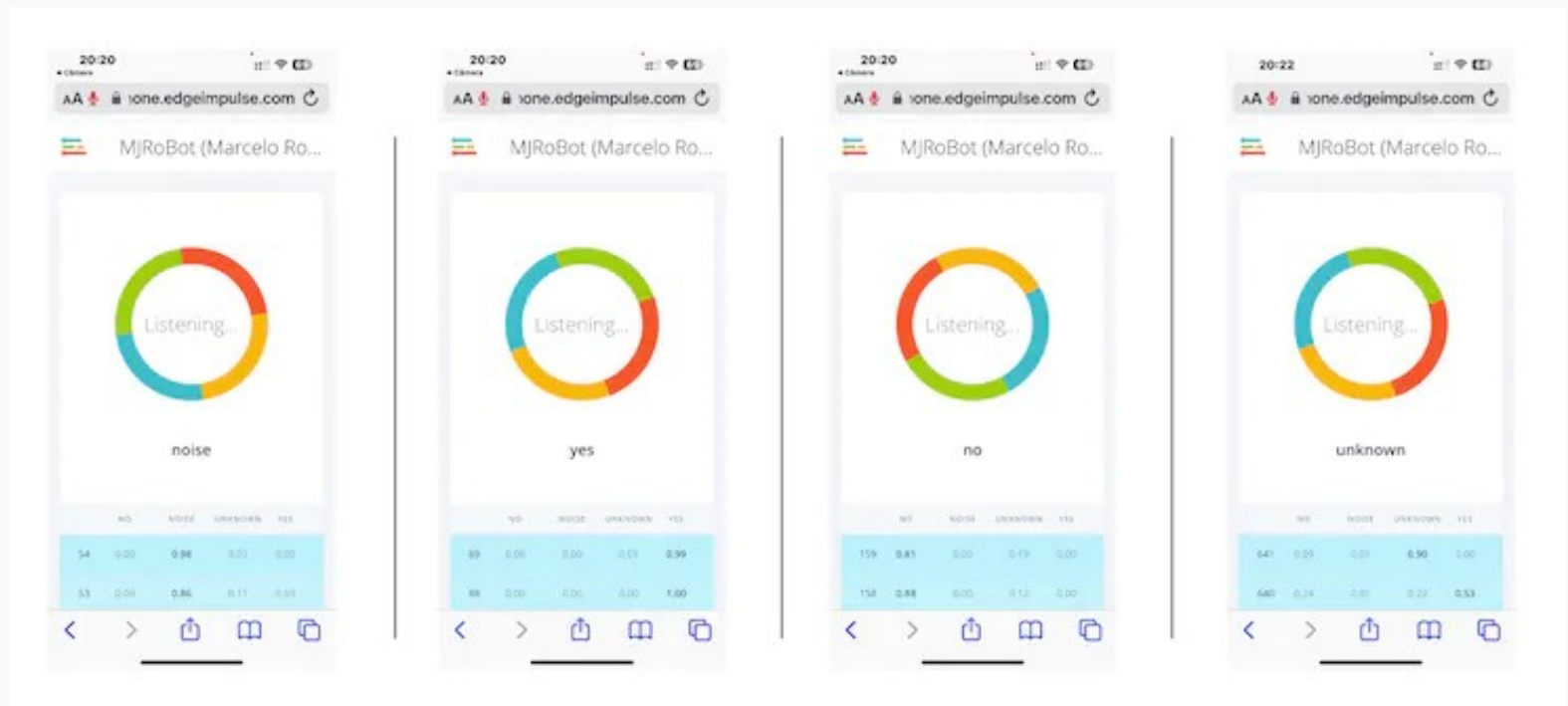
Sample length (ms.)

Classify existing data

- Point your phone to the QR code and select the link.



- Your phone will be connected to Edge Impulse Studio. Select the option **Classification** on the app, and when it is running, start testing your keywords, confirming that the model is working with live and real data:



- We now want to **deploy the model on the Xiao**. Edge Impulse will package all the needed libraries, preprocessing functions, and trained models, downloading them to your computer.
- Select the Deployment tab in Edge Impulse, select the Arduino Library option, then choose Quantized (Int8) from the bottom menu and press Build.

Q

Arduino library x

 **Arduino library**

An Arduino library with examples that runs on most Arm-based Arduino development boards.

MODEL OPTIMIZATIONS

Model optimizations can increase on-device performance but may reduce accuracy.

 **EON™ Compiler**

Quantized (int8)

Selected ✓

	IMAGE	CLASSIFIER	TOTAL
LATENCY	1 ms.	2.33 ms.	3.33 ms.
RAM	4.0K	182.8K	182.8K
FLASH	-	62.2K	-
ACCURACY			-

Unoptimized (float32)

Select

	IMAGE	CLASSIFIER	TOTAL
LATENCY	1 ms.	-	1 ms.
RAM	4.0K	-	4.0K
FLASH	-	-	-
ACCURACY			-

To compare model accuracy, run model testing for all available optimizations.

Run model testing

Estimate for Himax WiseEye2 (Cortex-M55 400 MHz) + Ethos-U55-64 · [Change target](#)

Build

- Now it is time for a real test. We will make inferences wholly disconnected from the Edge Impulse. Let's change one of the ESP32 code examples created when you deploy the Arduino Library.
- In your Arduino IDE, go to the File/Examples tab look for your project, and select esp32/esp32_microphone:



- This code was created for the ESP-EYE built-in microphone, which should be adapted for our device.
- Start changing the libraries to handle the I2S bus:

```
41 /* Includes -----  
42 #include <XIAO-ESP32S3-KWS_inferencing.h>  
43  
44 #include "freertos/FreeRTOS.h"  
45 #include "freertos/task.h"  
46  
47 #include "driver/i2s.h"  
48
```

To:

```
#include <I2S.h>  
#define SAMPLE_RATE 16000U  
#define SAMPLE_BITS 16
```


- Initialize the I2S microphone at setup(), including the lines:

```
void setup()
{
  ...
  I2S.setAllPins(-1, 42, 41, -1, -1);
  if (!I2S.begin(PDM_MONO_MODE,
SAMPLE_RATE, SAMPLE_BITS)) {
    Serial.println("Failed to initialize I2S!");
    while (1) ;
  }
  ...
}
```

- On the static void `capture_samples(void* arg)` function, replace the line 153 that reads data from I2S mic:

```
145 static void capture_samples(void* arg) {  
146  
147     const int32_t i2s_bytes_to_read = (uint32_t)arg;  
148     size_t bytes_read = i2s_bytes_to_read;  
149  
150     while (record_status) {  
151  
152         /* read data at once from i2s */  
153         i2s_read((i2s_port_t)1, (void*)sampleBuffer, i2s_bytes_to_read, &bytes_read, 100);  
154     }
```

To:

```
/* read data at once from i2s */  
esp_i2s::i2s_read(esp_i2s::I2S_NUM_0,  
                  (void*)sampleBuffer,  
                  i2s_bytes_to_read,  
                  &bytes_read, 100);
```

- On function `static bool microphone_inference_start(uint32_t n_samples)`, we should comment or delete lines 198 to 200, where the microphone initialization function is called. This is unnecessary because the I2S microphone was already initialized during the `setup()`.

- Finally, on static void microphone_inference_end(void) function, replace line 243:

```
241 static void microphone_inference_end(void)
242 {
243     i2s_deinit();
244     ei_free(inference.buffer);
245 }
```

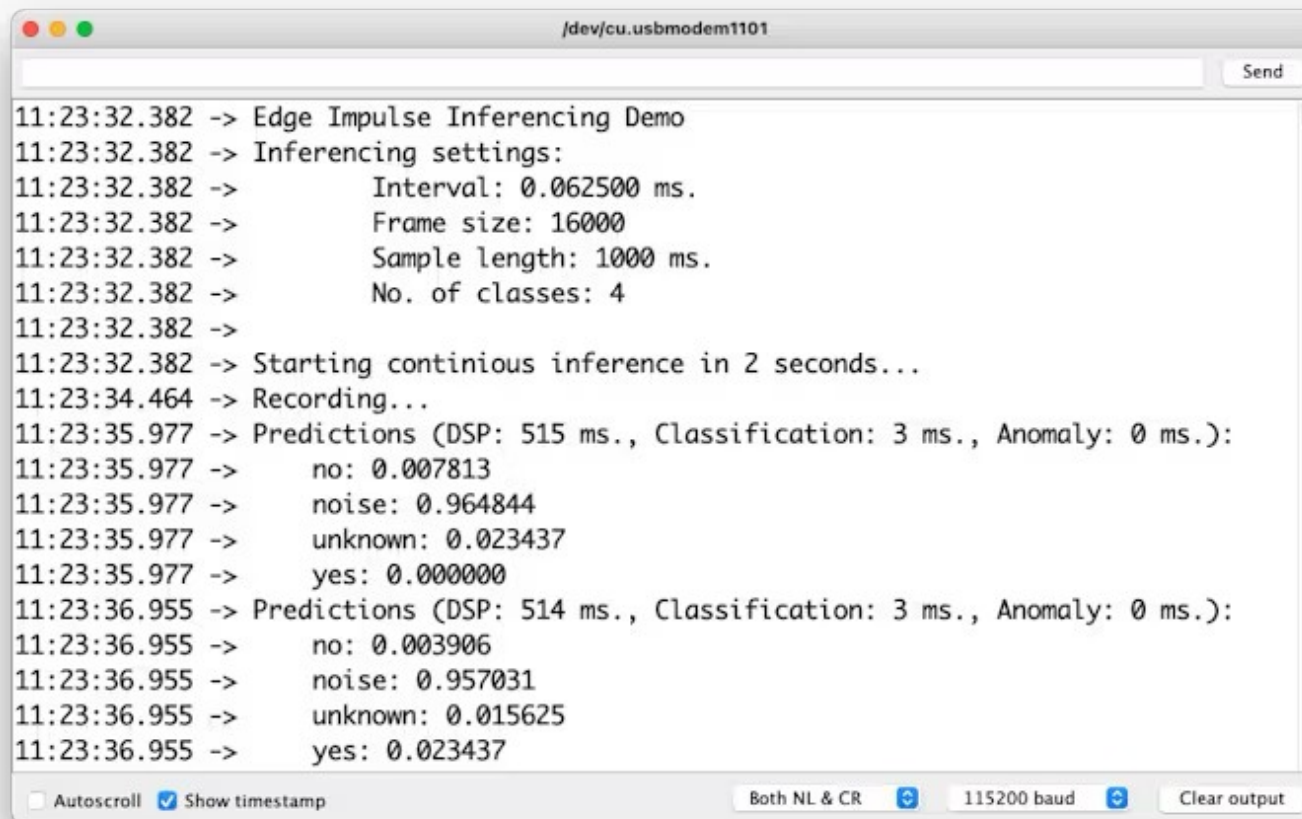
To:

```
static void microphone_inference_end(void)
{
    free(sampleBuffer);
    ei_free(inference.buffer);
}
```



- The Xiao ESP32S3 MUST have the PSRAM enabled. You can check it on the Arduino IDE upper menu: Tools-> PSRAM:OPI PSRAM
- The Arduino Library (esp32 by Espressif Systems should be version 2.017. Please do not update it.

Upload the code, Select Serial Monitor in the Arduino IDE and you should see:



```
/dev/cu.usbmodem1101
11:23:32.382 -> Edge Impulse Inferencing Demo
11:23:32.382 -> Inferencing settings:
11:23:32.382 ->     Interval: 0.062500 ms.
11:23:32.382 ->     Frame size: 16000
11:23:32.382 ->     Sample length: 1000 ms.
11:23:32.382 ->     No. of classes: 4
11:23:32.382 ->
11:23:32.382 -> Starting continious inference in 2 seconds...
11:23:34.464 -> Recording...
11:23:35.977 -> Predictions (DSP: 515 ms., Classification: 3 ms., Anomaly: 0 ms.):
11:23:35.977 ->     no: 0.007813
11:23:35.977 ->     noise: 0.964844
11:23:35.977 ->     unknown: 0.023437
11:23:35.977 ->     yes: 0.000000
11:23:36.955 -> Predictions (DSP: 514 ms., Classification: 3 ms., Anomaly: 0 ms.):
11:23:36.955 ->     no: 0.003906
11:23:36.955 ->     noise: 0.957031
11:23:36.955 ->     unknown: 0.015625
11:23:36.955 ->     yes: 0.023437
```

Autoscroll ☒ Show timestamp Both NL & CR 115200 baud Clear output

- Exercise:

- Use an existing dataset (Yes/No) instead of gathering your data 
- Switch on the LED when a specific word is detected 
- Gather Yes/No in your language or in Japanese. What accuracy can you reach?  
- Can you detect other sounds? Like the AC on or off?  

Thanks!

TinyML whispers low,
Code dances on micro souls,
AI blooms in small.

mzennaro@ictp.it